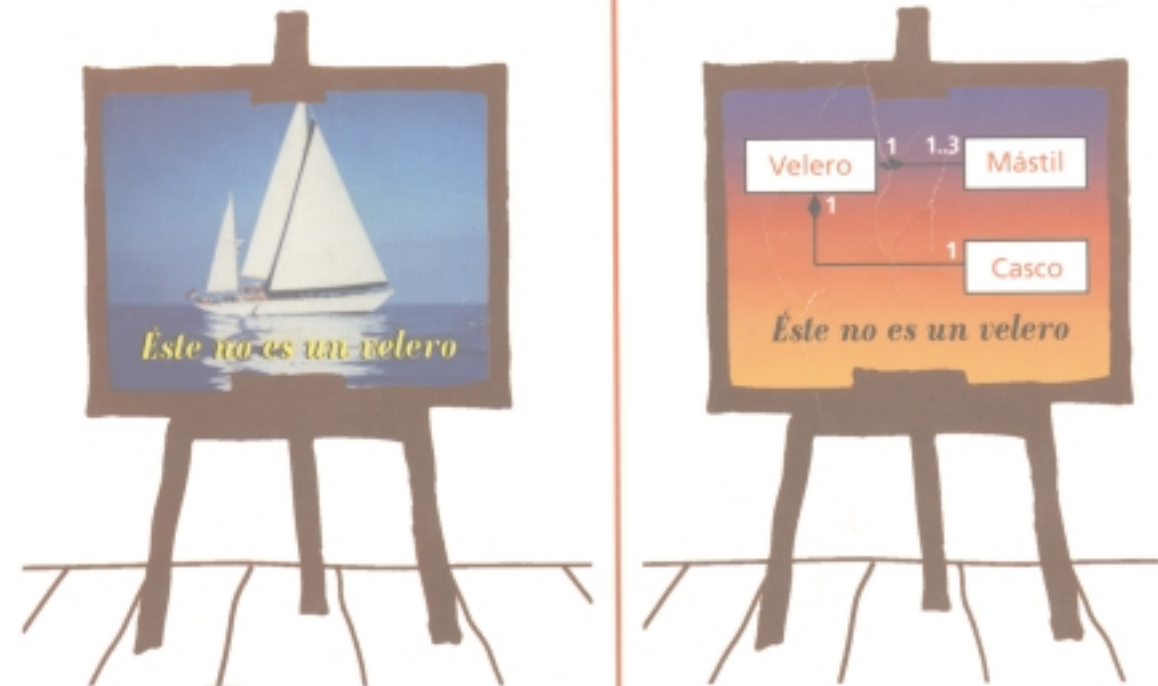


UML Y PATRONES

Introducción al análisis y diseño orientado a objetos



CRAIG LARMAN

PRENTICE
HALL

PEARSON

UML Y PATRONES

INTRODUCCIÓN AL ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CRAIG LARMAN

Versión en español:

Luz María Henández Rodríguez
Traductora profesional

Con revisión técnica de:

Ing. Humberto Cárdenas Anaya
*Director de la Carrera de Ingeniería en Sistemas Computacionales,
Profesor del Departamento de Sistemas de Información,
Instituto Tecnológico y de Estudios Superiores de Monterrey, Campus Estado de México*

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DPTO. DE DOCUMENTACION Y BIBLIOTECA
BIBLIOTECA CENTRAL
Ing. Edo. Garcia de Zuniga
MONTEVIDEO - URUGUAY

No. de Entrada 053681
27.3.01



PRENTICE
HALL

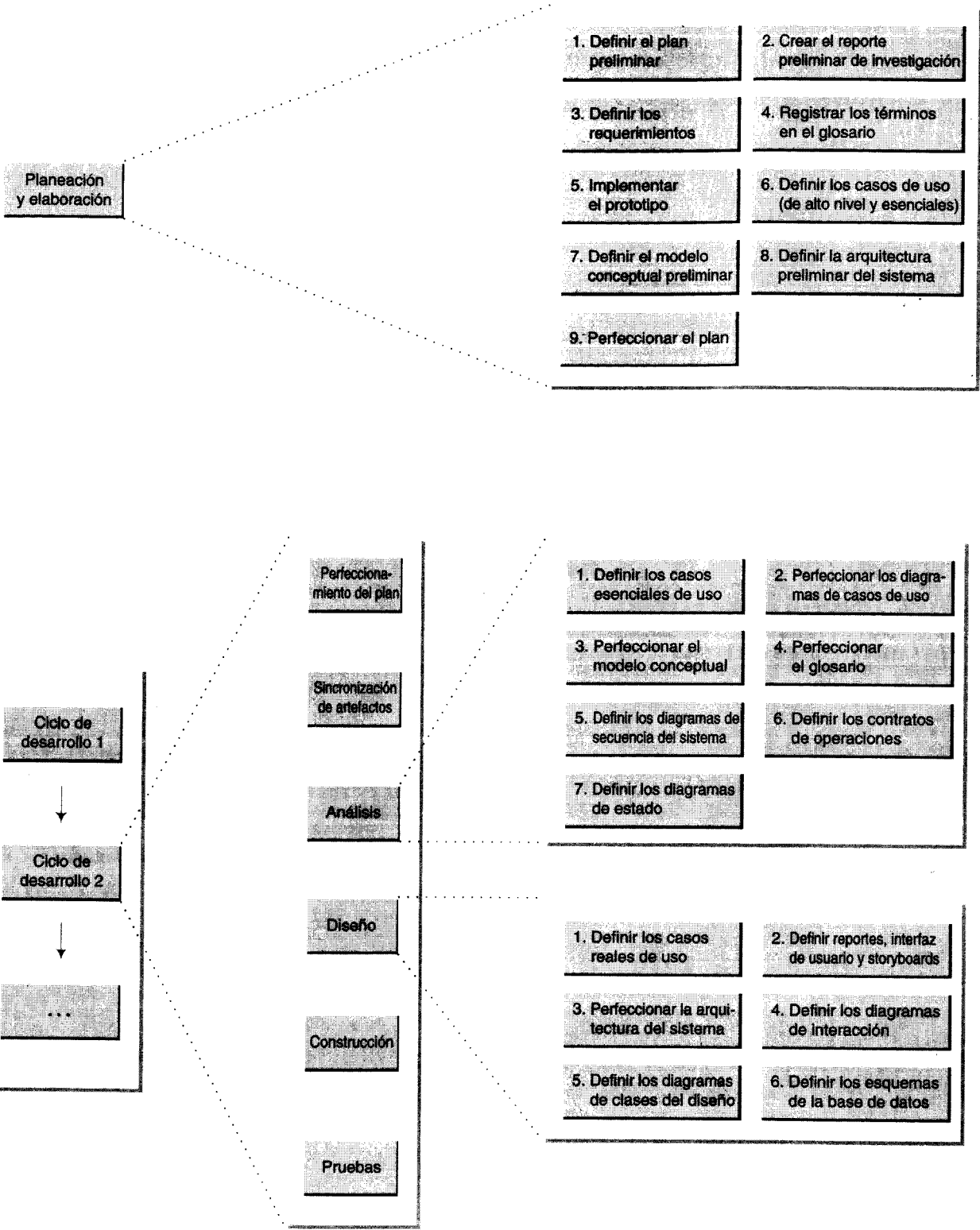
Addison
Wesley
Longman

MÉXICO • ARGENTINA • BOLIVIA • BRASIL • COLOMBIA • COSTA RICA • CHILE • ECUADOR
EL SALVADOR • ESPAÑA • GUATEMALA • HONDURAS • NICARAGUA • PANAMÁ
PARAGUAY • PERÚ • PUERTO RICO • REPÚBLICA DOMINICANA • URUGUAY • VENEZUELA

AMSTERDAM • HARLOW • MIAMI • MUNICH • NUEVA DELHI • MENLO PARK • NUEVA JERSEY
NUEVA YORK • ONTARIO • PARÍS • SINGAPUR • SYDNEY • TOKIO • TORONTO • ZURICH

SIBUR

Ejemplo de actividades de desarrollo



Patrones generales de software para asignar responsabilidades (GRASP)

Patrón	Descripción
Experto	¿Quién asumirá la responsabilidad en el caso general? Asignar una responsabilidad al experto en información: la clase que posee la información necesaria para cumplir con la responsabilidad.
Creador	¿Quién crea? Asignar a la clase B la responsabilidad de crear una instancia de clase A, si se cumple una de las siguientes condiciones: 1. B contiene A 2. B agrega A 3. B tiene los datos de inicialización de A 4. B registra A 5. B utiliza A muy de cerca
Controlador	¿Quién administra un evento del sistema? Asignar la responsabilidad de administrar un mensaje de eventos del sistema a una clase que represente una de las siguientes opciones: 1. El negocio o la organización global (un controlador de fachada). 2. El “sistema” global (un controlador de fachada). 3. Un ser animado del dominio que realice el trabajo (un controlador de papeles). 4. Una clase artificial (Fabricación Pura) que represente el caso de uso (un controlador de casos de uso).
Bajo Acoplamiento (evaluativo)	¿Cómo dar soporte a poca dependencia y a una mayor reutilización? Asignar las responsabilidades de modo que se mantenga bajo acoplamiento.
Alta Cohesión (evaluativa)	¿Cómo mantener controlable la complejidad? Asignar las responsabilidades de modo que se mantenga una alta cohesión?
Polimorfismo	¿Quién, cuándo el comportamiento varía según el tipo? Cuando varía el tipo (clase) de alternativas o comportamientos relacionados, asignar la responsabilidad del comportamiento —mediante operaciones polimórficas— a los tipos en que varía el comportamiento.
Fabricación Pura	¿Quién, cuándo uno está desesperado y no quiere violar los patrones Alta Cohesión ni Bajo Acoplamiento? Asignar un conjunto muy alto de cohesión de responsabilidades a una clase artificial que no represente nada en el dominio del problema, a fin de brindar soporte a una alta cohesión, a bajo acoplamiento y a la reutilización.
Indirección	¿Quién, para evitar el acoplamiento directo? Asignar la responsabilidad a un objeto intermedio para que medie entre otros componentes o servicios, de modo que no se acoplen directamente.
No Hables con Extraños (ley de Demeter)	¿Quién, para no conocer la estructura de los objetos indirectos? Asignar la responsabilidad al objeto directo del cliente para colaborar con el objeto indirecto, de modo que el cliente no necesita conocer el objeto indirecto. En el método, los mensajes únicamente pueden enviarse a los siguientes objetos: • El objeto <i>this</i> (o el <i>self</i>). • Un parámetro del método. • Un atributo de <i>self</i> . • Un elemento de una colección que sea un atributo de <i>self</i> . • Un objeto creado dentro del método.

RESUMEN DE CONTENIDO

PARTE I INTRODUCCIÓN 1

1	ANÁLISIS Y DISEÑO ORIENTADOS A OBJETOS	3
2	INTRODUCCIÓN A UN PROCESO DE DESARROLLO	17
3	DEFINICIÓN DE MODELOS Y ARTEFACTOS	29

PARTE II FASE DE PLANEACIÓN Y DE ELABORACIÓN 33

4	CASO DE ESTUDIO: EL PUNTO DE VENTA	35
5	CONOCIMIENTO DE LOS REQUERIMIENTOS	39
6	CASOS DE USO: DESCRIPCIÓN DE PROCESOS	47
7	CLASIFICACIÓN Y PROGRAMACIÓN DE LOS CASOS DE USO	73
8	INICIO DE UN CICLO DE DESARROLLO	81

PARTE III FASE DE ANÁLISIS (1) 83

9	CONSTRUCCIÓN DE UN MODELO CONCEPTUAL	85
10	MODELO CONCEPTUAL: AGREGACIÓN DE LAS ASOCIACIONES	105
11	MODELO CONCEPTUAL: AGREGACIÓN DE LOS ATRIBUTOS	119
12	REGISTRO DE LOS TÉRMINOS EN EL GLOSARIO	131
13	COMPORTAMIENTO DE LOS SISTEMAS: DIAGRAMA DE LA SECUENCIA DEL SISTEMA	135
14	COMPORTAMIENTO DE LOS SISTEMAS: CONTRATOS	145

PARTE IV FASE DE DISEÑO (1) 159

15	DEL ANÁLISIS AL DISEÑO	161
16	DESCRIPCIÓN DE LOS CASOS REALES DE USO	163
17	DIAGRAMAS DE COLABORACIÓN	167
18	GRASP: PATRONES PARA ASIGNAR RESPONSABILIDADES	185
19	DISEÑO DE UNA SOLUCIÓN CON OBJETOS Y PATRONES	217
20	DETERMINACIÓN DE LA VISIBILIDAD	247
21	DIAGRAMAS DE CLASES DEL DISEÑO	255
22	ALGUNOS ASPECTOS DEL DISEÑO DE SISTEMAS	271

PARTE V FASE DE CONSTRUCCIÓN (1) 293

23 MAPEO DE LOS DISEÑOS PARA CODIFICACIÓN 295
24 SOLUCIÓN EN PROGRAMA DE JAVA 309

PARTE VI FASE DE ANÁLISIS (2) 315

25 ELECCIÓN DE LOS REQUERIMIENTOS DEL CICLO DE DESARROLLO 2 317
26 CÓMO RELACIONAR CASOS MÚLTIPLES DE USO 321
27 EXTENSIÓN DEL MODELO CONCEPTUAL 329
28 GENERALIZACIÓN 335
29 PAQUETES: ORGANIZACIÓN DE LOS ELEMENTOS 349
30 REFINAMIENTO DEL MODELO CONCEPTUAL 355
31 MODELO CONCEPTUAL: RESUMEN 367
32 COMPORTAMIENTO DE LOS SISTEMAS 373
33 MODELADO DEL COMPORTAMIENTO EN LOS DIAGRAMAS DE ESTADO 379

PARTE VII FASE DE DISEÑO (2) 391

34 GRASP: MÁS PATRONES PARA ASIGNAR RESPONSABILIDADES 393
35 DISEÑO CON MÁS PATRONES 405

PARTE VIII TEMAS ESPECIALES 425

36 OTRA NOTACIÓN DE UML 427
37 PROBLEMAS DEL PROCESO DE DESARROLLO 433
38 ESQUEMAS, PATRONES Y PERSISTENCIA 455

APÉNDICE A. LECTURAS RECOMENDADAS 487

APÉNDICE B. EJEMPLOS DE ACTIVIDADES Y MODELOS DE DESARROLLO 489

BIBLIOGRAFÍA 495

GLOSARIO 497

ÍNDICE 503

CONTENIDO

PARTE I INTRODUCCIÓN 1

1 ANÁLISIS Y DISEÑO ORIENTADOS A OBJETOS 3
1.1 Aplicación del lenguaje UML y del análisis y el diseño orientados a objetos 3
1.2 Asignación de responsabilidades 5
1.3 ¿Qué son el análisis y el diseño? 6
1.4 ¿Qué son el análisis y el diseño orientados a objetos? 6
1.5 Una analogía: organización de la empresa MicroChaos 7
1.6 Un ejemplo del análisis y del diseño orientados a objetos 10
1.7 Comparación entre el análisis y el diseño orientados a objetos y los diseños orientados a funciones 14
1.8 Advertencia: el “análisis” y el “diseño” pueden provocar guerras terminológicas 14
1.9 El Unified Modeling Language, UML 15
2 INTRODUCCIÓN A UN PROCESO DE DESARROLLO 17
2.1 Introducción 17
2.2 El lenguaje UML y los procesos de desarrollo 19
2.3 Pasos de macronivel 20
2.4 Desarrollo iterativo 20
2.5 La fase de la planeación y de la elaboración 23
2.6 La fase de construcción: ciclos del desarrollo 25
2.7 Decidir cuándo crear artefactos 26
3 DEFINICIÓN DE MODELOS Y ARTEFACTOS 29
3.1 Introducción 29
3.2 Sistemas de construcción de modelos 29
3.3 Modelos muestra 30
3.4 Relación entre los artefactos 31

PARTE II FASE DE PLANEACIÓN Y DE ELABORACIÓN 33

4 CASO DE ESTUDIO: EL PUNTO DE VENTA 35
4.1 El sistema del punto de venta 35
4.2 Capas arquitectónicas y el énfasis en el caso de estudio 36
4.3 Nuestra estrategia: aprendizaje y desarrollo iterativos 36
5 CONOCIMIENTO DE LOS REQUERIMIENTOS 39
5.1 Introducción 39
5.2 Los requerimientos 41
5.3 Presentación general 41
5.4 Clientes 41
5.5 Metas 42
5.6 Funciones del sistema 42
5.7 Atributos del sistema 44
5.8 Otros artefactos en la fase de los requerimientos 46
6 CASOS DE USO: DESCRIPCIÓN DE PROCESOS 47
6.1 Introducción 47
6.2 Actividades y dependencias 49

6.3	Casos de uso	49
6.4	Actores	52
6.5	Un error común en los casos de uso	53
6.6	Identificación de los casos de uso	53
6.7	Caso de uso y procesos del dominio	54
6.8	Casos de uso, funciones del sistema y rastreabilidad	55
6.9	Diagramas de los casos de uso	55
6.10	Formatos de los casos de uso	55
6.11	Los sistemas y sus fronteras	56
6.12	Casos de uso primarios, secundarios y opcionales	58
6.13	Casos esenciales de uso comparados con los casos reales de uso	58
6.14	Sobre la notación	61
6.15	Casos de uso dentro de un proceso de desarrollo	63
6.16	Pasos del proceso en un sistema del punto de venta	64
6.17	Modelos muestra	71
7	CLASIFICACIÓN Y PROGRAMACIÓN DE LOS CASOS DE USO	73
7.1	Introducción	73
7.2	Programación de los casos de uso en los ciclos de desarrollo	75
7.3	Clasificación de los casos de uso en la aplicación al punto de venta	76
7.4	El caso de uso de arranque	76
7.5	Programación de los casos de uso en la aplicación del punto de venta	77
7.6	Versiones del caso de uso "Comprar productos"	78
7.7	Resumen	80
8	INICIO DE UN CICLO DE DESARROLLO	81
8.1	Inicio de un ciclo de desarrollo	81
PARTE III FASE DE ANÁLISIS (1) 83		
9	CONSTRUCCIÓN DE UN MODELO CONCEPTUAL	85
9.1	Introducción	85
9.2	Actividades y dependencias	87
9.3	Modelos conceptuales	87
9.4	Estrategias para identificar los conceptos	91
9.5	Conceptos idóneos para el dominio del punto de venta	94
9.6	Directrices para construir modelos conceptuales	96
9.7	Solución de los conceptos similares: comparación entre TPDV y Registro	97
9.8	Construcción de un modelo del mundo <i>irreal</i>	98
9.9	Especificación o descripción de conceptos	99
9.10	Definición de términos en el lenguaje UML	101
9.11	Modelos Patrón	103
10	MODELO CONCEPTUAL: AGREGACIÓN DE LAS ASOCIACIONES	105
10.1	Introducción	105
10.2	Asociaciones	105
10.3	Notación de las asociaciones en el UML	106
10.4	Identificación de las asociaciones: lista de asociaciones comunes	107
10.5	¿Qué grado de detalle deberían tener las asociaciones?	109
10.6	Directrices de las asociaciones	110
10.7	Papeles	110
10.8	Asignación de nombre a las asociaciones	111
10.9	Asociaciones múltiples entre dos tipos	112
10.10	Asociaciones e implementación	113
10.11	Asociaciones del dominio del punto de venta	113
10.12	Modelo conceptual del punto de venta	115

11	MODELO CONCEPTUAL: AGREGACIÓN DE LOS ATRIBUTOS	119
11.1	Introducción	119
11.2	Atributos	120
11.3	Notación de los atributos en el UML	120
11.4	Tipos de atributos válidos	120
11.5	Tipos de atributos no primitivos	124
11.6	Modelado de cantidades y unidades de los atributos	125
11.7	Atributos del sistema del punto de venta	126
11.8	Atributos en el modelo del punto de venta	127
11.9	Multiplicidad entre VentasLineadeProducto y Producto	128
11.10	Modelo conceptual del punto de venta	129
11.11	Conclusión	129
12	REGISTRO DE LOS TÉRMINOS EN EL GLOSARIO	131
12.1	Introducción	131
12.2	Glosario	131
12.3	Actividades y dependencias	132
12.4	Ejemplo de glosario aplicado al sistema del punto de venta	132
13	COMPORTAMIENTO DE LOS SISTEMAS: DIAGRAMA DE LA SECUENCIA DEL SISTEMA	135
13.1	Introducción	135
13.2	Actividades y dependencias	135
13.3	Comportamiento del sistema	137
13.4	Diagramas de la secuencia del sistema	137
13.5	Ejemplo de un diagrama de la secuencia de un sistema	137
13.6	Eventos y operaciones de un sistema	138
13.7	Cómo elaborar un diagrama de la secuencia de un sistema	140
13.8	Diagramas de la secuencia de un sistema y otros artefactos	140
13.9	Eventos y fronteras de un sistema	141
13.10	Asignación de nombre a los eventos y a las operaciones de un sistema	142
13.11	Presentación del texto del caso de uso	143
13.12	Modelos muestra	144
14	COMPORTAMIENTO DE LOS SISTEMAS: CONTRATOS	145
14.1	Introducción	145
14.2	Actividades y dependencias	145
14.3	Comportamiento de un sistema	147
14.4	Contratos	147
14.5	Ejemplo de contrato: introducirProducto	147
14.6	Secciones del contrato	148
14.7	Cómo preparar un contrato	149
14.8	Poscondiciones	150
14.9	El espíritu de las poscondiciones: el escenario y el telón	151
14.10	Explicación: poscondiciones de <i>introducirProducto</i>	152
14.11	¿Cuán completas deben ser las poscondiciones?	153
14.12	Descripción de los detalles y algoritmos del diseño: notas	153
14.13	Precondiciones	153
14.14	Recomendación sobre cómo redactar contratos	154
14.15	Contratos para el caso de uso <i>Comprar productos</i>	155
14.16	Contratos para el caso de uso <i>Inicio</i>	157
14.17	Cambios del modelo conceptual	158
14.18	Modelos muestra	158

PARTE IV FASE DE DISEÑO (1) 159

15	DEL ANÁLISIS AL DISEÑO	161
	15.1 Conclusión de la fase de análisis	161
	15.2 Inicio de la fase de diseño	162
16	DESCRIPCIÓN DE LOS CASOS REALES DE USO	163
	16.1 Introducción	163
	16.2 Actividades y dependencias	163
	16.3 Casos reales de uso	163
	16.4 Ejemplo: Comprar productos: versión 1	165
	16.5 Modelos muestra	166
17	DIAGRAMAS DE COLABORACIÓN	167
	17.1 Introducción	167
	17.2 Actividades y dependencias	167
	17.3 Diagramas de interacción	169
	17.4 Ejemplo de un diagrama de colaboración: efectuarPago	170
	17.5 Los diagramas de interacción son un artefacto de gran utilidad	170
	17.6 Éste es un capítulo dedicado exclusivamente a la notación	171
	17.7 Lea las directrices de diseño en los siguientes capítulos	171
	17.8 Cómo preparar diagramas de colaboración	172
	17.9 Notación básica de los diagramas de colaboración	173
	17.10 Modelos muestra	183
18	GRASP: PATRONES PARA ASIGNAR RESPONSABILIDADES	185
	18.1 Introducción	185
	18.2 Actividades y dependencias	187
	18.3 Los diagramas de interacción bien diseñados son muy útiles	187
	18.4 Responsabilidades y métodos	187
	18.5 Las responsabilidades y los diagramas de interacción	188
	18.6 Patrones	189
	18.7 GRASP: patrones de los principios generales para asignar responsabilidades	191
	18.8 La notación del UML para los diagramas de clase	192
	18.9 Experto	193
	18.10 Creador	197
	18.11 Bajo acoplamiento	200
	18.12 Alta cohesión	203
	18.13 Controlador	206
	18.14 Responsabilidades, representación de papeles y las tarjetas CRC	215
19	DISEÑO DE UNA SOLUCIÓN CON OBJETOS Y PATRONES	217
	19.1 Introducción	217
	19.2 Diagramas de interacción y otros artefactos	218
	19.3 Modelo conceptual del punto de venta	222
	19.4 Diagramas de colaboración para la aplicación TPDV	222
	19.5 El diagrama de colaboración: introducirProducto	223
	19.6 El diagrama de colaboración: terminarVenta	229
	19.7 El diagrama de colaboración: efectuarPago	233
	19.8 El diagrama de colaboración: Iniciar	238
	19.9 Cómo conectar la capa de presentación y la de dominio	243
	19.10 Resumen	245
20	DETERMINACIÓN DE LA VISIBILIDAD	247
	20.1 Introducción	247
	20.2 Visibilidad entre objetos	247
	20.3 Visibilidad	248
	20.4 Presentación de la visibilidad en el UML	253

21	DIAGRAMAS DE CLASES DEL DISEÑO	255
	21.1 Introducción	255
	21.2 Actividades y dependencias	255
	21.3 Cuándo crear diagramas de clases del diseño	257
	21.4 Ejemplo de un diagrama de clases del diseño	257
	21.5 Diagramas de clases del diseño	257
	21.6 Cómo elaborar un diagrama de clases del diseño	258
	21.7 Comparación entre el modelo conceptual y los diagramas de clases del diseño	259
	21.8 Creación de diagramas de clases del diseño para el punto de venta	259
	21.9 Notación de los detalles de los miembros	268
	21.10 Modelos muestra	270
	21.11 Resumen	270
22	ALGUNOS ASPECTOS DEL DISEÑO DE SISTEMAS	271
	22.1 Introducción	271
	22.2 Arquitectura clásica de tres capas	273
	22.3 Arquitecturas multicapas orientadas a objetos	274
	22.4 Cómo mostrar la arquitectura con paquetes de UML	275
	22.5 Identificación de los paquetes	278
	22.6 Estratos y particiones	278
	22.7 Visibilidad entre las clases de paquetes	279
	22.8 Interfaz de los paquetes de servicios: el patrón Fachada	280
	22.9 Sin visibilidad directa respecto a las ventanas: el patrón de Separación Modelo-Vista	281
	22.10 La comunicación indirecta en un sistema	284
	22.11 Coordinadores de las aplicaciones	287
	22.12 Almacenamiento y persistencia	290
	22.13 Modelos muestra	291

PARTE V FASE DE CONSTRUCCIÓN (1) 293

23	MAPEO DE LOS DISEÑOS PARA CODIFICACIÓN	295
	23.1 Introducción	295
	23.2 La programación y el proceso de desarrollo	295
	23.3 Mapeo de diseños para codificación	298
	23.4 Creación de las definiciones de clase a partir de los diagramas de clases del diseño	299
	23.5 Creación de métodos a partir de los diagramas de colaboración	302
	23.6 Actualizaciones de las definiciones de clases	305
	23.7 Las clases de contenedor/colección en código	306
	23.8 Manejo de las excepciones y de los errores	306
	23.9 Definición del método Venta-hacerLineadeProducto	307
	23.10 Orden de la implementación	307
	23.11 Resumen del mapeo del diseño a la codificación	308
24	SOLUCIÓN EN PROGRAMA DE JAVA	309
	24.1 Introducción a la solución convertida en programa	309

PARTE VI FASE DE ANÁLISIS (2) 315

25	ELECCIÓN DE LOS REQUERIMIENTOS DEL CICLO DE DESARROLLO 2	317
	25.1 Requerimientos del ciclo de desarrollo 2	317
	25.2 Suposiciones y simplificaciones	317

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

26 **CÓMO RELACIONAR CASOS MÚLTIPLES DE USO 321**
26.1 Introducción 321
26.2 Cuándo crear casos de uso independientes 321
26.3 Diagramas de casos de uso con las relaciones usa 322
26.4 Documentos de casos de uso con las relaciones usa 323

27 **EXTENSIÓN DEL MODELO CONCEPTUAL 329**
27.1 Nuevos conceptos en el sistema del punto de venta 329

28 **GENERALIZACIÓN 335**
28.1 Generalización 335
28.2 Definición de supertipos y de subtipos 336
28.3 Cuándo definir un subtipo 339
28.4 Cuándo definir un supertipo 342
28.5 Jerarquías de los tipos del punto de ventas 342
28.6 Tipos abstractos 345
28.7 Construcción de modelos con estados cambiantes 347
28.8 Jerarquías de clases y herencia 348

29 **PAQUETES: ORGANIZACIÓN DE LOS ELEMENTOS 349**
29.1 Introducción 349
29.2 Notación de los paquetes en el UML 350
29.3 Cómo partir el modelo conceptual 351
29.4 Paquetes del modelo conceptual del punto de venta 352

30 **REFINAMIENTO DEL MODELO CONCEPTUAL 355**
30.1 Introducción 355
30.2 Tipos asociativos 355
30.3 Agregación y composición 359
30.4 Nombres de los papeles de la asociación 362
30.5 Los papeles como conceptos y los papeles en las asociaciones 363
30.6 Elementos derivados 364
30.7 Asociaciones calificadas 365
30.8 Asociaciones recursivas o reflexivas 366

31 **MODELO CONCEPTUAL: RESUMEN 367**
31.1 Introducción 367
31.2 Paquete de los conceptos del dominio 368
31.3 Paquete básico/varios 368
31.4 Pagos 369
31.5 Productos 369
31.6 Ventas 370
31.7 Transacciones de autorización 371

32 **COMPORTAMIENTO DE LOS SISTEMAS 373**
32.1 Diagramas de secuencia del sistema 373
32.2 Nuevos eventos del sistema 374
32.3 Contratos 375

33 **MODELADO DEL COMPORTAMIENTO EN LOS DIAGRAMAS DE ESTADO 379**
33.1 Introducción 379
33.2 Eventos, estados y transiciones 379
33.3 Diagramas de estado 381
33.4 Diagramas de estado para los casos de uso 382
33.5 Diagramas de estado del sistema 383
33.6 Diagramas de estado de los casos de uso para la aplicación del punto de venta 384

33.7 Tipos que requieren diagramas de estado 384
33.8 Otros diagramas de estado para la aplicación Punto de Venta 386
33.9 Ejemplificación de eventos externos y de intervalos 387
33.10 Notación complementaria de los diagramas de estado 388

PARTE VII FASE DE DISEÑO (2) 391

34 **GRASP: MÁS PATRONES PARA ASIGNAR RESPONSABILIDADES 393**
34.1 GRASP: Patrones generales de software para asignar responsabilidades 393
34.2 Polimorfismo 394
34.3 Fabricación Pura 396
34.4 Indirección 398
34.5 No Hables con Extraños 400

35 **DISEÑO CON MÁS PATRONES 405**
35.1 Introducción 405
35.2 Estado (Pandilla de los Cuatro) 406
35.3 Polimorfismo (GRASP) 411
35.4 Singleton 413
35.5 Agente Remoto y Agente (Pandilla de los Cuatro) 416
35.6 El patrón Fachada y el Agente Dispositivo (Pandilla de los Cuatro) 418
35.7 El patrón Comando (Pandilla de los Cuatro) 421
35.8 Conclusión 424

PARTE VIII TEMAS ESPECIALES 425

36 **OTRA NOTACIÓN DE UML 427**
36.1 Introducción 427
36.2 Notación general 427
36.3 Interfaces 429
36.4 Diagramas de implementación 429
36.5 Mensajes asíncronos en los diagramas de colaboración 430
36.6 Interfaces de paquetes 432

37 **PROBLEMAS DEL PROCESO DE DESARROLLO 433**
37.1 Introducción 433
37.2 ¿Por qué molestarnos? 434
37.3 Directrices de un proceso eficiente 434
37.4 Desarrollo iterativo e incremental 435
37.5 Desarrollo orientado a los casos de uso 437
37.6 Énfasis inicial en la arquitectura 437
37.7 Fases del desarrollo 438
37.8 Duración de los ciclos de desarrollo 447
37.9 Aspectos del ciclo de desarrollo 448
37.10 Programación del desarrollo de las capas arquitectónicas 452

38 **ESQUEMAS, PATRONES Y PERSISTENCIA 455**
38.1 Introducción 455
38.2 El problema: objetos persistentes 456
38.3 La solución: un esquema de persistencia 456
38.4 ¿Qué es un esquema (framework)? 457
38.5 Requerimientos del esquema de persistencia 458
38.6 ¿Superclase ObjetoPersistente? 459
38.7 Ideas básicas 459

PREFACIO

38.8	Mapeo: patrón <i>Representación de objetos como tablas</i>	460
38.9	Identificación de objetos: el patrón <i>Identificador de objetos</i>	461
38.10	Intermediarios: el patrón <i>Intermediario de base de datos</i>	462
38.11	Diseño de esquemas: el patrón <i>Método de Plantilla</i>	463
38.12	Materialización: el patrón <i>Método de Plantillas</i>	464
38.13	Objetos colocados en espacio caché: el patrón <i>Administración de Caché</i>	467
38.14	Referencias inteligentes: los patrones <i>Agente Virtual</i> y <i>Puente</i>	468
38.15	Agentes Virtuales e Intermediarios de Bases de Datos	473
38.16	Cómo Representar las relaciones en tablas	475
38.17	El patrón <i>Instanciación de Objetos Complejos</i>	476
38.18	Operaciones de transacciones	479
38.19	Búsqueda de objetos en el almacenamiento persistente	483
38.20	Diseños alternos	484
38.21	Cuestiones sin resolver	486

APÉNDICE A. LECTURAS RECOMENDADAS	487
APÉNDICE B. EJEMPLOS DE ACTIVIDADES Y MODELOS DE DESARROLLO	489
BIBLIOGRAFÍA	495
GLOSARIO	497
ÍNDICE	503

Felicidades y gracias por leer este libro. Ponemos en sus manos una guía práctica y un mapa que lo conducirán por el camino del análisis y el diseño orientados a objetos. A continuación le explicamos para qué le servirá.

Diseño de sistemas de objetos robustos y de fácil mantenimiento.

Primero, como sigue proliferando la tecnología de objetos en el desarrollo del software, hoy aún más, gracias a la adopción generalizada de Java, el dominio del análisis y del diseño orientados a objetos es indispensable para usted si quiere crear sistemas robustos y de fácil mantenimiento orientados a objetos. Esa tecnología abre todo un mundo de oportunidades a los arquitectos, analistas y diseñadores.

Un mapa que lo guiará a través de los requerimientos, el análisis, el diseño y la codificación.

Segundo, si el análisis y el diseño orientados a objetos son un tema nuevo para el lector, seguramente querrá saber cómo avanzar por esta área tan compleja: nuestro libro ofrece un mapa de actividades bien definidas, para que vaya dominando gradualmente los requisitos de la codificación.

Uso de la notación UML para ejemplificar los modelos de análisis y diseño.

Tercero, el UML (*Unified Modeling Language*, Lenguaje Unificado de Construcción de Modelos) nació como una notación estándar de la construcción de modelos; por ello le será de gran utilidad familiarizarse con él. En este libro, mediante la notación de UML, aprenderá las técnicas del análisis y el diseño orientados a objetos.

Mejora de los diseños aplicando los patrones de diseño de la “pandilla de los cuatro” (gang-of-four, GoF) y GRASP.

Cuarto, los patrones de diseño ofrecen las soluciones e idiomas “más prácticos” de que los expertos en el diseño orientado a objetos se sirven para crear sistemas. En este libro aprenderá los patrones del diseño, entre ellos los famosos patrones de la pandilla de los cuatro y, sobre todo, los de GRASP, que comunican los principios fundamentales de la asignación de responsabilidades en el diseño orientado a objetos. Al ir aprendiendo y utilizando los patrones, dominará más rápidamente el análisis y el diseño.

Aprendizaje eficiente basado en una exposición muy bien organizada.

Quinto, la estructura y el enfoque de este libro se fundan en muchos años de experiencia en la capacitación y de tutoría en el arte del análisis y diseño orientados a objetos. En el libro se plasma esa experiencia docente: ofrece un enfoque refinado, probado y eficiente que facilita el aprendizaje del tema, con lo cual seguramente los lectores aprovecharán al máximo el tiempo que dediquen a la lectura y al aprendizaje.

*Aprendizaje
mediante un
ejercicio realista.*

*Traducción
a código.*

*Diseño de una
arquitectura
de capas.*

*Diseño de un
esquema.*

Sexto, se examina a fondo un solo caso de estudio, con el propósito de explicar realista e íntegramente el proceso del análisis y el diseño orientados a objetos, profundizando además en detalles complejos del problema: es éste un ejercicio muy parecido a los casos que se encuentran en la realidad.

Séptimo, se muestra cómo mapear los elementos del diseño orientado a objetos para codificarlos en Java.

Octavo, explica cómo diseñar una arquitectura de capas (o niveles) y relaciona la capa gráfica de la interfaz con las del dominio y los servicios del sistema. Se trata de un tema de importancia práctica que a menudo pasa inadvertido.

Finalmente, se muestra al lector cómo diseñar un esquema orientado a objetos, que además es aplicado específicamente a la creación de un esquema de almacenamiento persistente en una base de datos.

Objetivos

El objetivo global es:

Ayudar a los estudiantes y diseñadores a crear mejores diseños orientados a objetos, recurriendo para ello a principios explicables y a la heurística.

Al estudiar y aplicar la información y los métodos del libro, al lector le será más fácil entender un problema a partir de sus procesos y conceptos, así como diseñar una solución sólida por medio de objetos.

A quiénes está destinado el libro

Este libro se dirige a los siguientes lectores:

- Diseñadores con experiencia en un lenguaje de programación orientado a objetos, pero con pocos conocimientos —o relativamente pocos— en el área del análisis y el diseño orientados a objetos.
- Estudiantes de computación o que toman cursos de ingeniería de software en los cuales se enseña la tecnología de objetos.
- Quienes estén familiarizados con el análisis y el diseño orientados a objetos y que deseen aprender la notación del (*Unified Modeling Language*, Lenguaje Unificado de Construcción de Modelos) y aplicar patrones, o bien que quieran ampliar y perfeccionar sus habilidades de análisis y diseño.

Requisitos

Se suponen y se requieren algunos conocimientos previos para aprovechar mejor este libro:

- Conocimiento y experiencia en un lenguaje de programación orientado a objetos, como C++, Java o Smalltalk.
- Conocimiento de los conceptos fundamentales de la tecnología de objetos: clase, instancia, interfaz, polimorfismo, encapsulamiento y herencia.

No se definen en este libro los conceptos básicos de la tecnología de objetos.

Organización del libro

La organización del libro se basa en la siguiente estrategia global: introducir los temas del análisis y el diseño orientados a objetos en un orden semejante al de un proyecto de desarrollo de software que se realiza a través de dos ciclos de desarrollo iterativo. En el primero se describen el análisis y el diseño. En el segundo, se exponen nuevos temas de análisis y diseño, mientras que los temas actuales se estudian más a fondo.

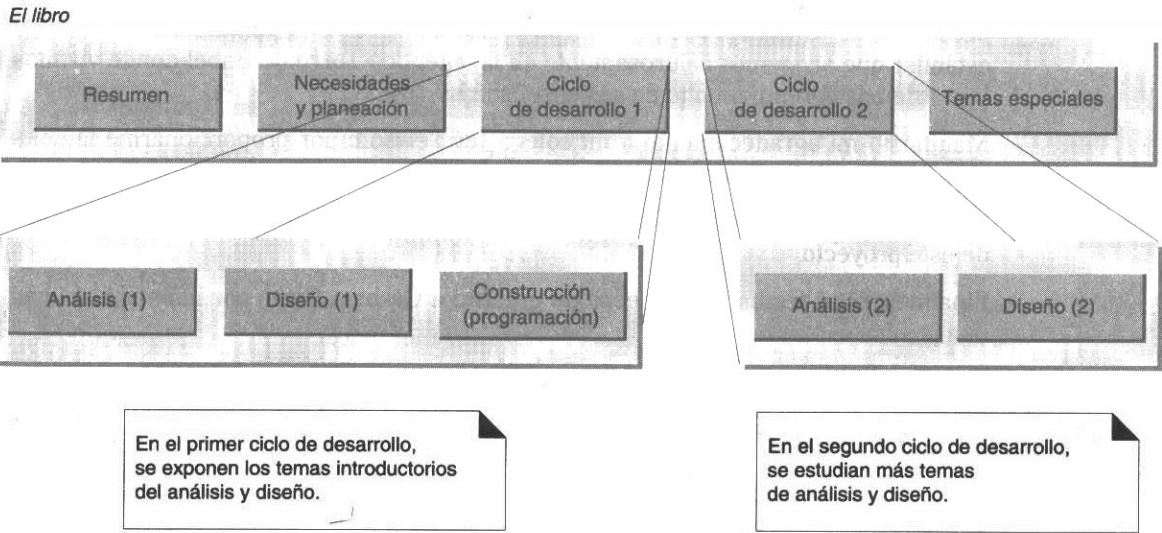


Figura 1. El libro está organizado como un proyecto de desarrollo.

Objetivo del libro

La tecnología de objetos es un área muy prometedora, pero su potencial no se aprovechará plenamente si no se poseen las habilidades apropiadas. Me propongo difundir su adopción eficiente mediante la práctica del análisis y del diseño orientados a objetos y favorecer al mismo tiempo la adquisición de las destrezas relacionadas, porque me he dado cuenta de que son indispensables para la creación y el mantenimiento exitosos de los sistemas importantes.

Reconocimientos

Agradezco a todos los usuarios y estudiosos del programa UML y del análisis y diseño orientados a objetos a quienes he procurado brindar mi ayuda; ellos son mis mejores maestros.

Un testimonio especial de gratitud merecen los revisores del libro (o de parte de él), entre ellos: Kent Beck, Jens Coldewey, Clay Davis, Tom Heruska, Luke Hohmann, David Norris, David Nunn, Brett Schuchert y todo el equipo Mercury. Doy gracias a Grady Booch por revisar el tutorial de la edición en inglés y a Jim Rumbaugh por sus comentarios sobre la relación existente entre el lenguaje UML y el proceso.

Agradezco sus excelentes comentarios acerca del proceso y los modelos a Todd Girvin, John Hebley, Tom Heruska, David Norris, David Nunn, Charles Rego y Raj Wall.

Mi más profunda gratitud a Grady Booch, Ivar Jacobson y Jim Rumbaugh por el desarrollo del *Unified Modeling Language*; por la creación de una notación abierta y estándar que acogemos calurosamente en la auténtica Torre de Babel donde vivimos hoy. Además aprendí mucho de sus enseñanzas.

Manifiesto mi agradecimiento a mi colega Jef Newsom por proporcionarme la solución en Java al estudio de casos.

Gracias a Paul Becker, mi editor de Prentice-Hall, por su inquebrantable fe en el valor de este proyecto.

Finalmente, un testimonio especial de gratitud a Graham Glass por haberme abierto una puerta.

Semblanza del autor

Craig Larman tiene la licenciatura y la maestría en computación, y desde 1978 ha venido desarrollando sistemas grandes y pequeños de software en plataformas que abarcan desde macrocomputadoras hasta microcomputadoras, valiéndose de tecnologías de software que incluyen desde 4GLs hasta programación lógica y programación orientada a objetos.

A principios de los años ochenta, Larman se enamoró de la inteligencia artificial y de las técnicas de programación de los sistemas del conocimiento, área donde tuvo su primer contacto con la programación orientada a objetos (en Lisp). Enseñó y trabajó con la programación en Lisp desde 1984, con Smalltalk desde 1986, con C++ desde 1991 y recientemente con Java, además de impartir varios métodos de análisis y diseño orientados a objetos. Ha ayudado a más de 2,000 estudiantes en el aprendizaje de temas relacionados con la tecnología de objetos.

Actualmente es jefe de instructores en ObjectSpace, compañía que se especializa en el cómputo distribuido, en agentes y en tecnología de objetos.

Si el lector lo desea, puede ponerse en contacto con Craig por correo electrónico: clarman@acm.org.

Convenciones tipográficas

Se utilizan las negritas cuando se presenta un **nuevo término** en una oración.

Se emplean las cursivas para el nombre de una *Clase* o *método*.

Las referencias bibliográficas se hacen escribiendo entre corchetes el apellido del autor y el año de publicación de la obra [Bob67]. En la bibliografía aparecen los datos completos de cada obra.

Se adoptó el criterio de no usar acentos en los modelos conceptuales, diagramas y programas.

Con el operador “-.” de resolución del ámbito independiente del lenguaje se indican un método y su clase relacionada así: *ClassName-methodName()*.

PARTE I INTRODUCCIÓN

ANÁLISIS Y DISEÑO ORIENTADOS A OBJETOS

Objetivos

- Comparar y contrastar el análisis y el diseño.
- Definir el análisis y el diseño orientados a objetos.
- Relacionar por analogía el análisis y el diseño orientado a objetos con el fin de organizar una empresa.

1.1 Aplicación del lenguaje UML y del análisis y el diseño orientados a objetos

Desarrolle habilidades en el análisis y diseño orientados a objetos.

¿Qué significa contar con un sistema orientado a objetos que esté bien diseñado? En este libro ayudamos a programadores y estudiantes para que adquieran y utilicen habilidades prácticas en el análisis y el diseño orientados a objetos. Esas destrezas son indispensables para crear sistemas de software bien diseñados, robustos y de fácil mantenimiento, utilizando la tecnología de objetos y los lenguajes de programación orientados a objetos como C++, Java o Smalltalk.

El proverbio “El hábito no hace al monje” se aplica perfectamente a la tecnología de objetos. El hecho de conocer un lenguaje orientado a objetos (Java, por ejemplo) y además tener acceso a una rica biblioteca (como la de Java) es un primer paso necesario pero insuficiente para crear sistemas de objetos. Se requiere además analizar y diseñar un sistema desde la perspectiva de los objetos.

Nota del editor: En esta edición se adoptó el criterio de no usar acentos en los modelos conceptuales, diagramas y programas.

La práctica del análisis y el diseño orientados a objetos se explica con un caso de estudio relativamente exhaustivo que vamos desarrollando a lo largo del libro: ambos aspectos se profundizan lo suficiente para que se traten y resuelvan muchos de los detalles más engorrosos que plantea un problema realista.

“UML” son las siglas de **Unified Modeling Language** (Lenguaje Unificado de Construcción de Modelos), notación (esquemática en su mayor parte) con que se construyen sistemas por medio de conceptos orientados a objetos. En este libro ayudaremos a los programadores para que aprendan la notación de UML y la apliquen a un estudio de casos.

¿Cómo deberían asignarse las responsabilidades a las clases de objetos? ¿Cómo deberían interactuar éstos? ¿Qué papel debe destinársele a cada clase? Éstas son preguntas muy importantes cuando se diseña un sistema. Algunas soluciones ya probadas y eficaces de los problemas de diseño pueden expresarse (y se han plasmado) como un conjunto de principios, con heurística o **patrones** (fórmulas de solución de problemas que codifican los principios aceptados del diseño). En este libro enseñamos los patrones y así favorecemos el aprendizaje rápido y la utilización hábil de los tecnicismos fundamentales de esta tecnología.

Debido a las numerosas actividades que posiblemente exijan las condiciones durante la implementación, ¿cómo debería proceder el programador o equipo de programadores? En esta obra presentamos un *ejemplo* del **proceso de desarrollo** que describe un orden posible de actividades y un ciclo de vida del desarrollo. Pero no prescribe un proceso ni un método definitivos; se limita a ofrecer una muestra de pasos comunes.

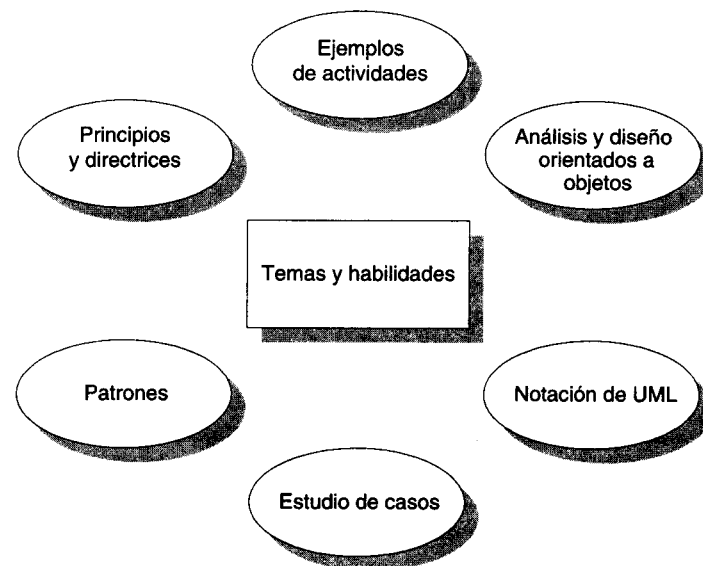


Figura 1.1 Temas y habilidades que se incluyen en el libro.

En conclusión, en este libro se ayuda al programador:

- A aplicar los principios y patrones para aprender a realizar mejores diseños.
- A efectuar varias actividades comunes en el análisis y en el diseño.
- A crear elementos útiles en la notación de UML.

Todo lo anterior lo ejemplificamos dentro del contexto de un estudio de casos.

1.2 Asignación de responsabilidades

La asignación de responsabilidades a los componentes es la habilidad más importante del diseño.

Hay muchas posibles actividades y elementos en el análisis y en el diseño, así como gran cantidad de principios y directrices. Supóngase que tuviéramos que elegir una sola habilidad práctica entre todos los temas aquí expuestos: una habilidad tan solitaria como una “isla desierta” por así decirlo. ¿Cuál sería?

La habilidad más importante en el análisis y el diseño orientados a objetos es asignar eficientemente las responsabilidades a los componentes del software.

¿Por qué? Porque es la actividad que debe efectuarse (es ineludible) y que influye más profundamente en la solidez, en la capacidad de mantenimiento y en la reutilizabilidad de los componentes del software.

El segundo lugar por orden de importancia aparece la obtención de los objetos o las abstracciones adecuadas. Ambos aspectos son decisivos: ponemos de relieve la asignación de responsabilidades porque tiende a ser más difícil de dominar.

En un objeto real, un programador tal vez no tenga la oportunidad de efectuar alguna otra actividad relacionada con el análisis o con el diseño: el proceso de desarrollo “con prisa por codificar”. Pero incluso en tales casos es inevitable la asignación de responsabilidades.

Por ello, en la fase de diseño de este libro ponemos de relieve los principios de la asignación de responsabilidades y ofrecemos las herramientas que le ayudarán al programador a dominarla.

Nueve principios fundamentales de la asignación de responsabilidades, codificados en los patrones de GRASP, se describen y se aplican para ayudarle al lector a dominar este aspecto.

Lo anterior no significa que el resto de las actividades carezcan de importancia. Por ejemplo, es indispensable crear un modelo conceptual que identifique los conceptos (objetos) en el dominio del problema. Pero, en cuanto habilidad de tipo “isla desierta”, el hecho de saber asignar responsabilidades es lo que en definitiva hace o destruye a un sistema.

1.3 ¿Qué son el análisis y el diseño?

Análisis: investigación.

Para crear una aplicación de software hay que describir el problema y las necesidades o requerimientos: en qué consiste el conflicto y qué debe hacerse. El **Análisis** se centra en una *investigación* del problema, no en la manera de definir una solución. Por ejemplo, si se desea un nuevo sistema de información computarizada de una biblioteca, ¿cuáles procesos de la institución se relacionan con su uso?

Diseño: solución.

Para desarrollar una aplicación, también es necesario contar con descripciones detalladas y de alto nivel de la solución lógica y saber cómo satisface los requerimientos y las restricciones. El **Diseño** pone de relieve una *solución lógica*: cómo el sistema cumple con los requerimientos. He aquí un ejemplo: ¿de qué manera el software del sistema de información de la biblioteca capturará y registrará los préstamos de libros? En definitiva, los diseños se implementan en software y en hardware.

1.4 ¿Qué son el análisis y el diseño orientados a objetos?

Análisis orientado a objetos: ¿conceptos?

La esencia del **análisis y el diseño orientados a objetos** consiste en situar el dominio de un problema y su solución lógica dentro de la perspectiva de los objetos (cosas, conceptos o entidades), como se advierte en la figura 1.2.

Diseño orientado a objetos: ¿objetos de software?

Durante el **análisis orientado a objetos** se procura ante todo identificar y describir los objetos —o conceptos— dentro del dominio del problema. Por ejemplo, en el caso del sistema de información de la biblioteca, algunos de los conceptos son *Libro*, *Biblioteca* y *Cliente*.

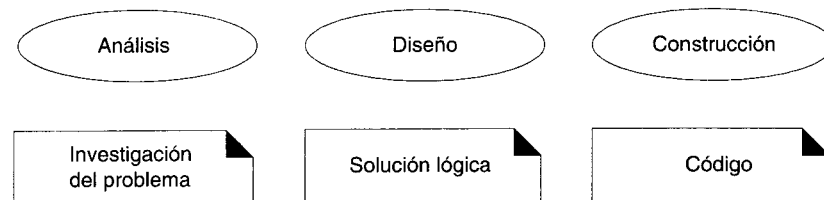
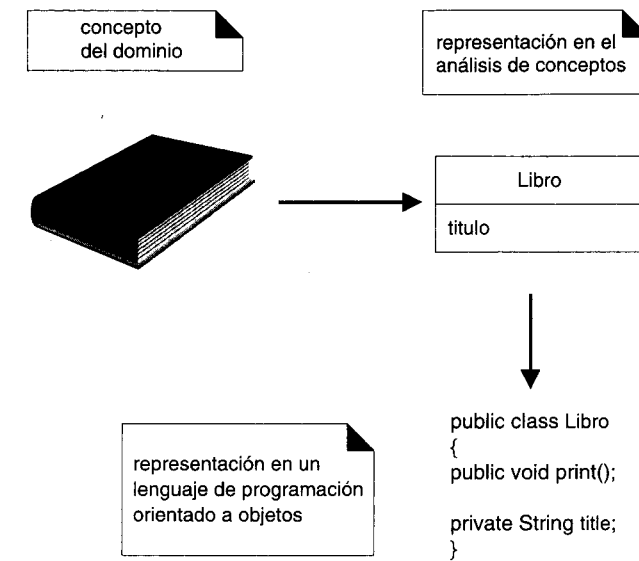


Figura 1.2 Significado de las actividades de desarrollo.

Durante el **diseño orientado a objetos**, se procura definir los objetos lógicos del software que finalmente serán implementados en un lenguaje de programación orientado a objetos. Los objetos tienen atributos y métodos. Así, en el sistema de la biblioteca un objeto de software *Libro* puede tener un atributo *título* y un método *imprimir* (figura. 1.3).

Finalmente, durante la **construcción o programación orientada a objetos**, se implementan los componentes del diseño, como una clase *Libro* en C++, Java, Small-talk o Visual Basic.



UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

Figura 1.3 La orientación a objetos se centra en la representación de objetos.

1.5 Una analogía: organización de la empresa MicroChaos

La organización de una empresa y el análisis y el diseño orientados a objetos guardan semejanzas.

Algunos de los principales pasos del análisis y del diseño orientados a objetos pueden explicarse por analogía con la forma en que se organiza una compañía.

1.5.1 MicroChaos está creciendo rápidamente...

Imagine ser el fundador y director general de MicroChaos, compañía recién creada y especializada en el software que aplica los modelos matemáticos de la teoría del caos al análisis del mercado accionario (esto ya se ha hecho en el mundo de las finanzas). Su producto principal es *MicroButterfly*.



En el inicio tan prometedor de la compañía todo mundo compartía el trabajo: contestaban el teléfono, surtían los pedidos, escribían los programas de computación. La compañía había alcanzado un éxito extraordinario: había muchos empleados de ingreso reciente y el

personal se había vuelto dispersivo porque realizaba demasiadas tareas diferentes. Ha llegado, pues, el momento de implantar una organización más rigurosa. ¿Cómo procedería usted, en su calidad de director general, para organizar a los empleados y las actividades?

1.5.2 ¿Qué son los procesos de negocios?

El primer paso consiste en analizar lo que debe hacer una empresa —sus procesos de negocios— si quiere seguir funcionando: realizar ventas, pagar a empleados y acreedores, desarrollar programas de computación.

Desde el punto de vista del método del análisis y del diseño orientados a objetos, este paso nos recuerda el **análisis de requerimientos**, en el cual los procesos y las necesidades de los negocios se descubren y se expresan en los **casos de uso**. Los casos son descripciones narrativas textuales de los procesos de una empresa o sistema, como se muestra en seguida:

Caso de uso: Colocar un pedido.

Descripción: Este caso de uso comienza cuando un cliente telefona a un representante de ventas para hacer una compra de MicroButterfly. El representante anota en una nueva orden la información relativa al cliente y al producto.

Identificar los procesos y registrarlos en los casos de uso no es en realidad una actividad del análisis orientado a objetos; de ninguna manera se centra en objetos.

Con todo, se trata de un paso importante y generalizado en los métodos del análisis y diseño orientados a objetos. Asimismo, los casos de uso forman parte del lenguaje UML. A continuación mostramos al lector gráficamente nuestra analogía:

Analogía de la empresa	Análisis y diseño orientados a objetos	Documentos relacionados
¿Cuáles son los procesos de negocios?	Análisis de requerimientos	Casos de uso

1.5.3 ¿Cuáles son los papeles o las funciones en la organización?

El siguiente paso consiste en identificar los papeles de las personas que intervendrán en los procesos: cliente, representante de ventas, ingeniero de software, etcétera.

En la perspectiva del análisis y del diseño orientados a objetos, este paso nos recuerda al **análisis del dominio orientado a objetos** que expresamos con un **modelo conceptual**. Este modelo presenta las diversas categorías de las cosas en el dominio; no sólo los papeles de las personas sino también todas las cosas de interés, según se observa en la figura 1.4.

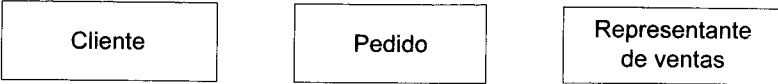


Figura 1.4 Conceptos en la empresa.

Analogía de la empresa	Análisis y diseño orientados a objetos	Documentos relacionados
¿Cuáles son los procesos de negocios?	Análisis de requerimientos	Casos de uso
¿Cuáles son los papeles de los empleados?	Análisis del dominio	Modelo conceptual

1.5.4 ¿Qué funciones cumple cada empleado?
¿Cómo colabora el personal?

Una vez identificados los procesos de su empresa y el personal, es el momento de determinar la manera de cumplir los procesos. Se trata de una actividad de diseño, o sea orientada a las soluciones. Junto con los empleados, defina las responsabilidades de ellos a fin de efectuar las tareas necesarias para llevar a cabo un proceso. También necesita definir de qué manera los empleados colaborarán o compartirán el trabajo.



Desde el punto de vista del análisis y del diseño orientados a objetos, esta actividad se parece al **diseño orientado a objetos** que pone de relieve la **asignación de responsabilidades**. La asignación significa distribuir las funciones y las responsabilidades entre varios objetos de software en la aplicación, del mismo modo que se asignan a los papeles de los empleados. Los objetos de software normalmente colaboran o interactúan para cumplir con sus responsabilidades, como lo hacen las personas.

La descripción de la asignación de las responsabilidades y las interacciones de objetos a menudo se expresan gráficamente con **diagramas de diseño de clase** y con **diagramas de colaboración**; unos y otros muestran la definición de clases y el flujo de mensajes entre los objetos de software.

Analogía de la empresa	Análisis y diseño orientados a objetos	Documentos relacionados
¿Cuáles son los procesos del negocio?	Análisis de requerimientos	Casos de uso
¿Cuáles son los papeles de los empleados?	Análisis del dominio	Modelo conceptual
¿Qué funciones cumplen los empleados? ¿Cómo interactúan ellos?	Asignación de responsabilidades, diseño de interacciones	Diagramas de diseño de clases, diagramas de colaboración

1.6 Un ejemplo del análisis y del diseño orientados a objetos

Un simple ejemplo nos permite ver todo el panorama.

Se necesita abundante información para explicar cabalmente el análisis y el diseño orientados a objetos. Para no perdernos en muchos detalles, ofrecemos al lector una explicación sucinta sobre algunos de los pasos y diagramas usando para ello un ejemplo simple: un “juego de dados” en que un jugador lanza dos dados. Si el total es siete, gana y de lo contrario pierde.

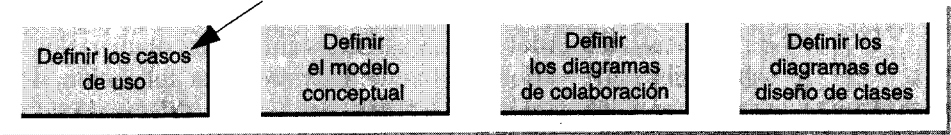


Las notaciones utilizadas forman parte del lenguaje UML.

Observe, por favor, que en el ejemplo no están representados todos los pasos posibles ni los diagramas; tan sólo aparecen los más comunes y esenciales.

1.6.1 Definición de los casos de uso

Para entender los requerimientos se necesita, en parte, conocer los procesos del dominio y el ambiente externo, o sea los factores externos que participan en los procesos. Dichos procesos de dominio pueden expresarse en **casos de uso**, o sea, en descripciones narrativas de los procesos del dominio en un formato estructurado de prosa.



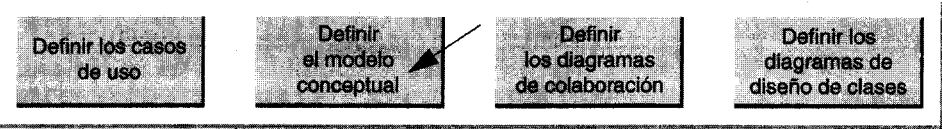
Los casos de uso no son propiamente un elemento del análisis orientado a objetos; se limitan a describir procesos y pueden ser igualmente eficaces en un proyecto de tecnología no orientada a objetos. No obstante, constituyen un paso preliminar muy útil porque describen las especificaciones de un sistema.

Por ejemplo, en el juego de dados el caso de uso de *Juega un Juego*.

Caso de uso:	Juega un Juego.
Participantes:	Jugador.
Descripción:	Este caso de uso comienza cuando el jugador recoge y hace rodar los dados. Si los puntos suman siete, gana y pierde si suman cualquier otro número.

1.6.2 Definición de un modelo conceptual

El análisis orientado a objetos tiene por finalidad estipular una especificación del dominio del problema y los requerimientos desde la perspectiva de la clasificación por objetos y desde el punto de vista de entender los términos empleados en el dominio. Para descomponer el dominio del problema hay que identificar los conceptos, los atributos y las asociaciones del dominio que se juzgan importantes. El resultado puede expresarse en un **modelo conceptual**, el cual se muestra gráficamente en un grupo de diagramas que describen los conceptos (objetos).



Por ejemplo, como se aprecia en la figura 1.5, en el dominio del juego de dados, una parte del modelo conceptual muestra los conceptos *Jugador*, *Dados* y *JuegodeDados*, sus asociaciones y atributos.

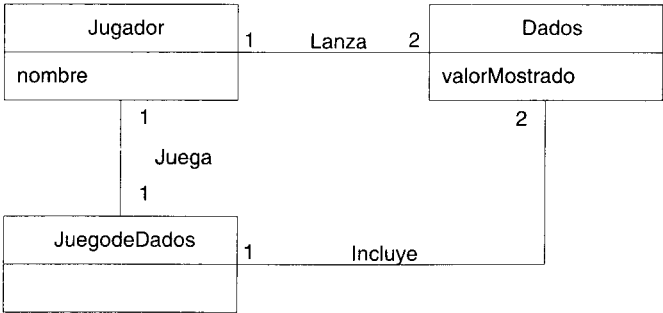


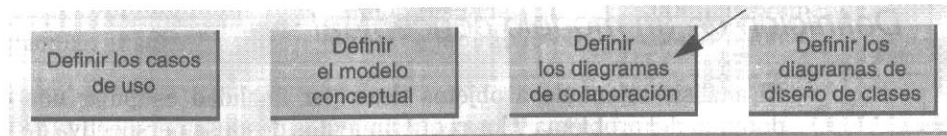
Figura 1.5 Modelo conceptual del juego de dados.

El modelo conceptual no es una descripción de los componentes del software; representa los conceptos en el dominio del problema en el mundo real.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

1.6.3 Definición de los diagramas de colaboración

El diseño orientado a objetos tiene por objeto definir las especificaciones lógicas del software que cumplan con los requisitos funcionales, basándose en la descomposición por clases de objetos. Un paso esencial de esta fase es la asignación de responsabilidades entre los objetos y mostrar cómo interactúan a través de mensajes, expresados en **diagramas de colaboración**. Estos presentan el flujo de mensajes entre las instancias y la invocación de métodos.



Por ejemplo, supongamos que se desea una simulación en el software del juego de dados. El diagrama de colaboración de la figura 1.6 muestra gráficamente el paso esencial del juego, enviando mensajes a las instancias de las clases *Jugador* y *Dado*.

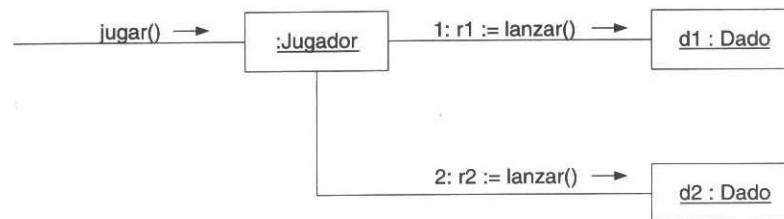


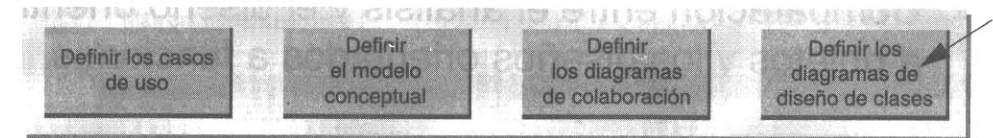
Figura 1.6 Diagrama de colaboración que muestra los mensajes entre los objetos de software.

1.6.4 Definición del diseño de clases

Para definir una clase es preciso contestar varias preguntas:

- ¿Cómo se conectan unos objetos a otros?
- ¿Cuáles son los métodos de una clase?

Si el lector quiere contestar las preguntas anteriores, examine detenidamente los diagramas de colaboración que indican las conexiones necesarias entre objetos, y también los métodos que cada clase de software debe definir. El **diagrama de diseño de clases** es el que expresa esos detalles. Muestra las definiciones de clase que han de implementarse en el software.



Por ejemplo, en el juego de dados, al examinar el diagrama de colaboración, obtenemos el siguiente diagrama del diseño de clases. Puesto que un mensaje *juego* se envía a una instancia *Jugador*, *Jugador* requiere un método *jugar*, mientras que *Dado* requiere un método *lanzar*.

A diferencia del modelo conceptual, este diagrama no muestra gráficamente conceptos del mundo real; describe únicamente los componentes del software.

Para indicar de qué manera los objetos se conectan entre sí a través de atributos, una línea con una flecha en la punta indicará un atributo. Por ejemplo, *JuegodeDados* posee un atributo que apunta a una instancia de un *Jugador*.

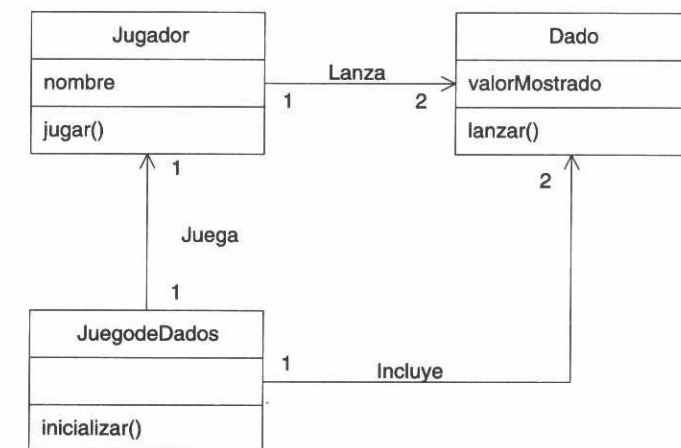


Figura 1.7 Diagrama del diseño de clases para los componentes de software.

El diagrama del diseño de clases de la figura 1.7 no está completo; representa únicamente el inicio de las especificaciones del software que se requieren para definir cabalmente cada clase.

1.6.5 Resumen del ejemplo del juego de dados

El juego de dados es un problema muy simple, que incluimos para centrarnos en algunos de los pasos y artefactos del análisis y diseño orientados a objetos y no en el dominio del problema. En capítulos subsecuentes trataremos más a fondo esos tres aspectos, sirviéndose para ello de un problema más complejo y realista.

1.7 Comparación entre el análisis y el diseño orientados a objetos y los diseños orientados a funciones

Los proyectos de software son complejos, y la estrategia primaria para superar la complejidad es la descomposición (divide y vencerás): dividir el problema en unidades manejables. Antes del advenimiento del análisis y diseño orientados a objetos, el método usual de descomponer un problema eran el **análisis y el diseño estructurados** cuya dimensión de descomposición es fundamentalmente por función o proceso, lo cual origina una división jerárquica de procesos constituidos por subprocesos.

Sin embargo, existen también otras dimensiones de descomposición; el análisis y el diseño orientados a objetos buscan ante todo descomponer un espacio de problema por objetos y no por funciones, como se advierte en la figura 1.8.

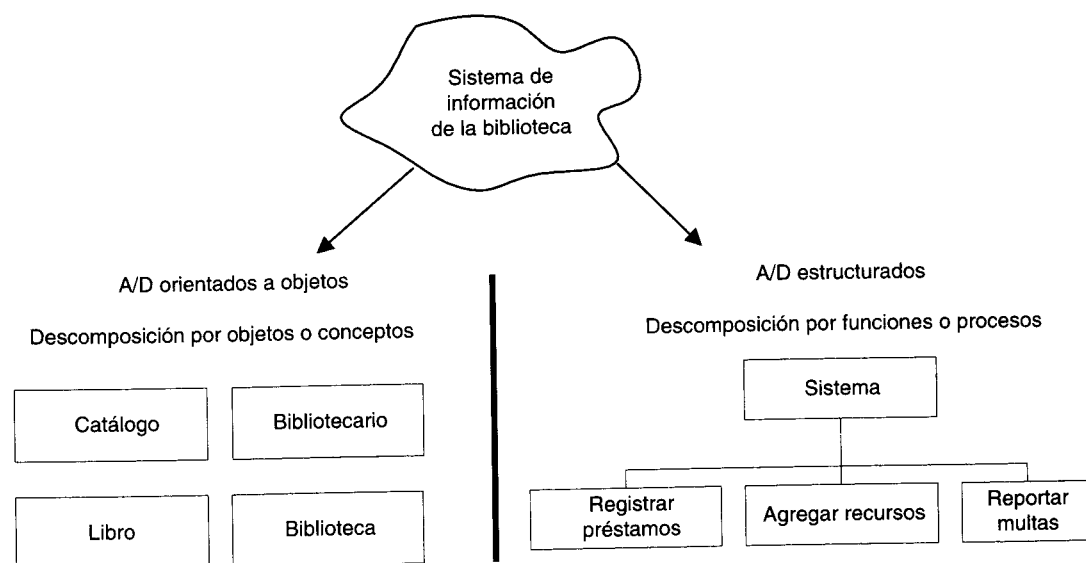


Figura 1.8 Comparación entre la descomposición orientada a objetos y la descomposición orientada a funciones.

1.8 Advertencia: el “análisis” y el “diseño” pueden provocar guerras terminológicas

La división entre análisis y diseño es poco clara; el trabajo de los dos existe en un continuo (figura 1.9), y los profesionales de los métodos del “análisis” y “diseño” clasifican una actividad en varios puntos del continuo. De ahí que no convenga adoptar una actitud rígida ante lo que constituye un paso del análisis o del diseño.

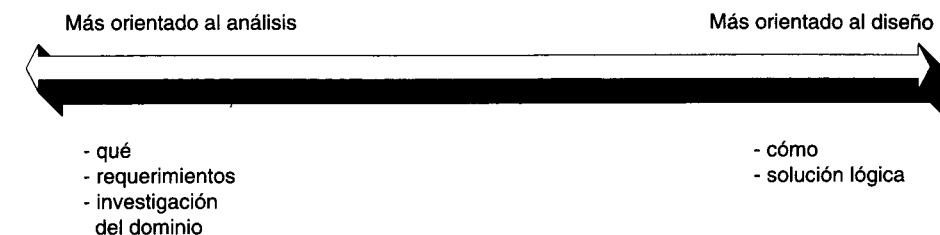


Figura 1.9 Las actividades de análisis y diseño existen en un continuo.

Debatir sobre la definición no resulta constructivo en absoluto, ya que las personas les dan distintos significados a esos dos términos y no se emplean con el mismo sentido en los métodos: lo importante es saber resolver el problema de crear software y darle mantenimiento con mayor eficiencia, con mayor rapidez y a un precio menor.

Con todo, en la práctica conviene contar con una distinción constante entre *investigación* (análisis) y *solución* (diseño), porque es útil tener un paso bien definido que indague la naturaleza del problema antes de buscar la manera de crear una solución. Además genera expectativas de un comportamiento apropiado entre los miembros del equipo; por ejemplo, durante el análisis esperan subrayar la *comprensión* del problema, posponen las cuestiones concernientes a la solución, al desempeño y a otros aspectos.

1.9 El Unified Modeling Language, UML

El UML (Lenguaje Unificado para la Construcción de Modelos) se define como un “lenguaje que permite especificar, visualizar y construir los artefactos de los sistemas de software...” [BJR97]. Es un sistema notacional (que, entre otras cosas, incluye el significado de sus notaciones) destinado a los sistemas de modelado que utilizan conceptos orientados a objetos.

El UML es un estándar incipiente de la industria para construir modelos orientados a objetos. Nació en 1994 por iniciativa de Grady Booch y Jim Rumbaugh para combinar sus dos famosos métodos: el de Booch y el OMT (*Object Modeling Technique*, Técnica de Modelado de Objetos). Más tarde se les unió Ivar Jacobson, creador del método OOSE (*Object-Oriented Software Engineering*, Ingeniería de Software Orientada a Objetos). En respuesta a una petición de OMG (*Object Management Group*, asociación para fijar los estándares de la industria) para definir un lenguaje y una notación estándar del lenguaje de construcción de modelos, en 1997 propusieron el UML como candidato.

Prescindiendo de la aceptación que pueda tener, este lenguaje recibió la aprobación *de facto* en la industria, pues sus creadores representan métodos muy difundidos de la primera generación del análisis y diseño orientados a objetos. Muchas organizaciones dedicadas al desarrollo de software y los proveedores de herramientas de CASE (Computer Aided Software Engineering) lo adoptaron, y muy probablemente se con-

vertirá en el estándar mundial que utilizarán los desarrolladores, los autores y los proveedores de herramientas de CASE.

Este libro no incluye todos los detalles de UML, un conjunto relativamente amplio de notaciones. Se centra en los diagramas de uso frecuente y en las características que más se utilizan dentro de ellos.

Si el lector desea un estudio exhaustivo de la notación UML en sus versiones iniciales, puede consultar la especificación completa en el sitio de Web de Rational Corporation: www.rational.com

Hay por los menos 10 notaciones diferentes de los elementos del análisis y diseño orientados a objetos; ello dificulta una comunicación eficaz y uniforme, la capacidad de aprender y las herramientas de CASE. Los autores del lenguaje UML —Booch, Jacobson y Rumbaugh— hicieron un excelente servicio a la comunidad de la tecnología de objetos al crear un lenguaje estandarizado de modelado que es elegante, expresivo y flexible.

El UML es un lenguaje para construir modelos; no guía al desarrollador en la forma de *realizar* el análisis y diseño orientados a objetos ni le indica cuál proceso de desarrollo adoptar.

En consecuencia, los metodólogos seguirán definiendo técnicas, modelos y procesos de desarrollo para la creación eficaz de sistemas de software; sólo que ahora pueden hacerlo en un lenguaje común: el UML.

INTRODUCCIÓN A UN PROCESO DE DESARROLLO

Objetivos

- Explicar el motivo del orden de los capítulos posteriores, que siguen los pasos del proceso que se describen en este capítulo.
- Seguir un proceso simple de desarrollo desde los requerimientos hasta la implementación.

2.1 Introducción

Un **proceso de desarrollo de software** es un método de organizar las actividades relacionadas con la creación, presentación y mantenimiento de los sistemas de software.

En este capítulo se da una muy breve introducción a las actividades fundamentales de un proceso, ofreciendo una guía de los pasos principales. Hemos incluido aquí esta explicación por dos motivos:

- En los capítulos subsecuentes se abordan el análisis y el diseño orientados a objetos a partir de esas actividades.
- Al seguir un proceso similar al que presentamos aquí, se sientan las bases para crear un proyecto de desarrollo manejable, reproducible y exitoso.

En el capítulo 37 encontrará el lector más temas concernientes al proceso.

2.1.1 Proceso y modelos que se recomiendan

El lenguaje UML no define un proceso estándar. Sus creadores admiten la importancia de contar con un lenguaje y un proceso muy sólidos para la construcción de modelos. Ofrecen su recomendación de lo que constituye un proceso adecuado en publicaciones aparte de las dedicadas al UML, porque la estandarización del proceso rebasaba entonces el ámbito de la definición del lenguaje.

Tampoco se prescribe en este libro un estándar o proceso nuevo.

Más importante que seguir un proceso o método oficiales es que el desarrollador adquiera habilidades que le permitan crear un buen diseño, y que las organizaciones favorezcan este tipo de desarrollo de destrezas. Esto se logra dominando varios principios y heurísticos que permiten identificar y abstraer los objetos idóneos, asignándoles después responsabilidades.

En este libro se ofrece una visión bastante representativa de los mejores métodos, de las actividades y modelos básicos en un proceso de desarrollo para sistemas orientados a objetos que se funden en un enfoque iterativo e incremental de los casos de uso. Los pasos y modelos recomendados constituyen, más que una fórmula, un *recorrido* por el panorama de desarrollo de software por medio de la tecnología de objetos. Estas actividades se incluyen como punto de partida para exponer, experimentar y crear un proceso adecuado a las necesidades concretas de la empresa del lector.

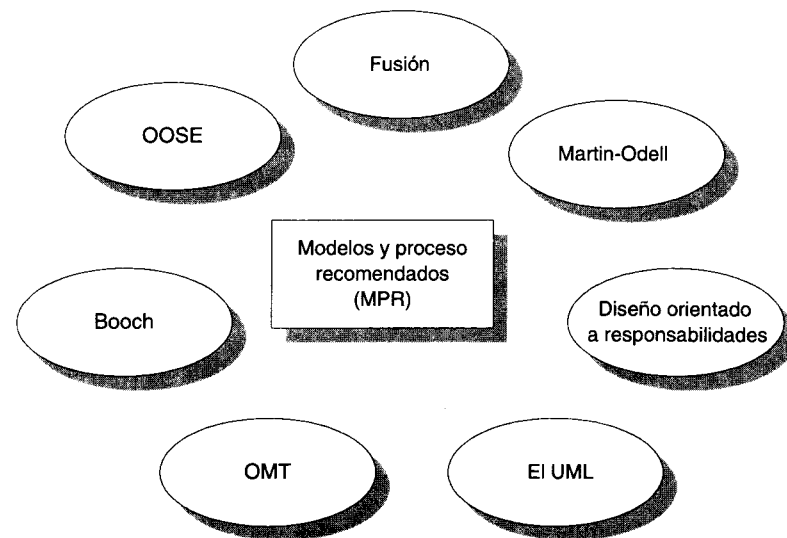


Figura 2.1 Factores que influyen en el proceso y los modelos recomendados en el libro.

No se trata de un método nuevo, sino la descripción de un proceso y de modelos generalmente recomendados (MPR) que utilizan los profesionales y que, con diversos nombres y ligeras modificaciones, se incluyen en otros métodos del análisis y diseño orientado a objetos [Rumbaugh97]. Para simplificar la explicación, en ocasiones utilizaremos las siglas *MPR* (modelos y proceso recomendados) y también para recalcar la naturaleza cíclica del desarrollo iterativo.

Los factores que más han influido en los MPR aquí descritos (figura 2.1) son la experiencia personal del autor en el desarrollo, el trabajo dedicado a ObjectSpace, el propio UML, Fusion [Coleman94], Martin-Odell [MO95], Responsibility-Driven Design [Wirfs-Brock90], OOSE (Objectory) [Jacobson92], OMT [Rumbaugh91] y Booch [Booch94].

2.1.2 Alcance

La descripción de un proceso incluye fundamentalmente las actividades que abarcan desde los requerimientos hasta la presentación o entrega. Además un proceso completo aborda puntos más amplios relacionados con la industrialización del desarrollo de software: ciclo de vida de un producto a largo plazo, documentación, soporte y capacitación, trabajo en paralelo y coordinaciones entre los participantes.¹ En esta introducción se ponen de relieve sólo las actividades básicas, sin entrar en muchos detalles.

En nuestro examen, se prescindirá de algunos pasos y aspectos esenciales del proceso: concepción, planeación, interacción de equipos en paralelo, administración del proyecto, documentación y realización de pruebas.

Omitiremos los pasos anteriores porque son ajenos al propósito fundamental del libro —aprender y aplicar habilidades en el análisis y diseño orientados a objetos— o porque los temas son demasiado extensos como para investigarlos de manera satisfactoria.

2.2 El lenguaje UML y los procesos de desarrollo

El lenguaje UML estandariza los artefactos y la notación, pero no define un proceso oficial de desarrollo. He aquí algunas de las razones que explican esto:

1. Aumentar las probabilidades de una aceptación generalizada de la *notación* estándar del modelado, sin la obligación de adoptar un proceso oficial.
2. La esencia de un proceso apropiado admite mucha variación y depende de las habilidades del personal, de la razón investigación-desarrollo, de la naturaleza del problema, de las herramientas y de muchos otros factores.

¹ Jacobson distingue entre *método*, que abarca los pasos individuales y secuenciales del desarrollo, y *proceso*, que amplía el método para aplicarlo a cuestiones más amplias de industrialización y de desarrollo en equipo [Jacobson92]. Aunque ésta es una distinción importante, no subrayaré la distinción para no complicar aún más un área ya saturada de tecnicismos.

Una vez aclaradas estas razones, procedemos a aplicar los principios generales y los pasos normales que guían un proceso eficaz.

2.3 Pasos de macronivel

En un nivel alto, los pasos principales en la presentación de una aplicación son los siguientes (figura 2.2):

1. **Planeación y elaboración:** planear, definir los requerimientos, construir prototipos, etcétera.
2. **Construcción:** la creación del sistema.
3. **Aplicación:** la transición de la implementación del sistema a su uso.

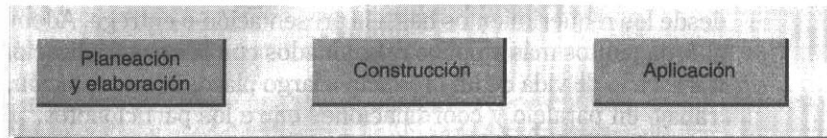


Figura 2.2 Pasos de macronivel en el desarrollo.

2.4 Desarrollo iterativo

Un ciclo de vida iterativo se basa en el agrandamiento y perfeccionamiento secuencial de un sistema a través de *múltiples* ciclos de desarrollo de análisis, diseño, implementación y pruebas.

El sistema crece al incorporar nuevas funciones en cada ciclo de desarrollo. Tras una fase preliminar de planeación y especificación, el desarrollo pasa a la fase de construcción a través de una serie de ciclos de desarrollo.

En cada ciclo se aborda un conjunto relativamente pequeño de requerimientos, pasando por el análisis, el diseño, la construcción y las pruebas (figura 2.3). El sistema va creciendo con cada ciclo que concluye.

Esto contrasta con el ciclo clásico de la vida en cascada, en el cual las actividades (análisis y diseño, entre otras) se llevan a cabo una vez con todos los requerimientos del sistema.

Entre las ventajas del desarrollo iterativo figuran las siguientes:

- La complejidad nunca resulta abrumadora.
- Se produce retroalimentación en una etapa temprana, porque la implementación se efectúa rápidamente con una parte pequeña del sistema.

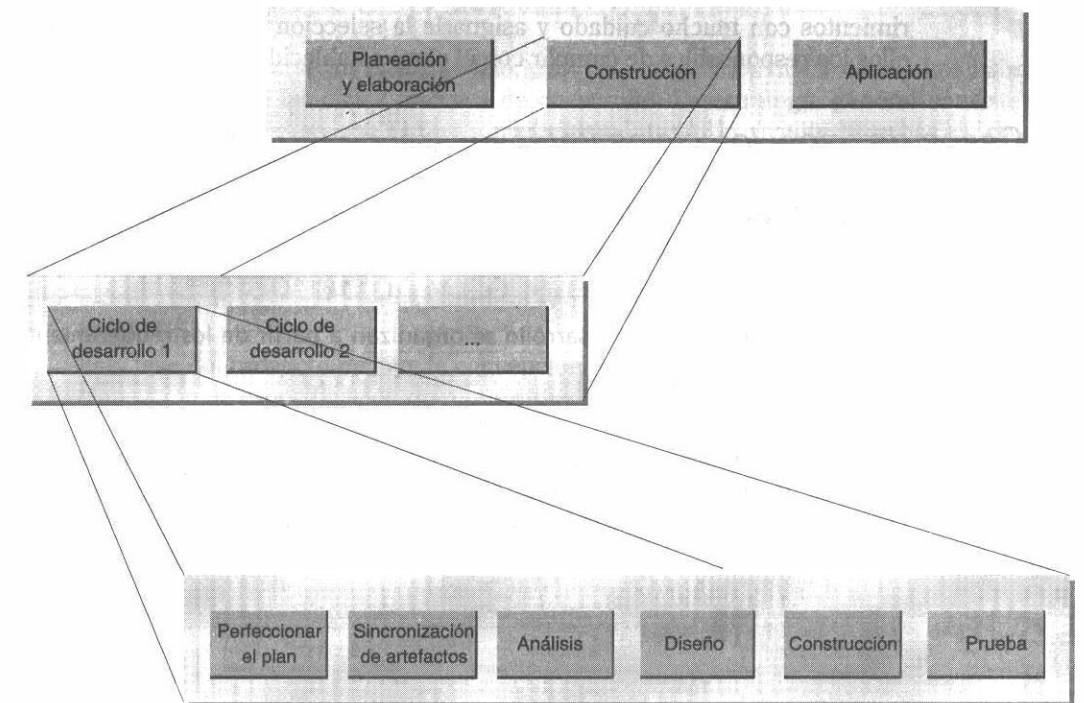


Figura 2.3 Ciclos iterativos de desarrollo.

2.4.1 Fijación de la duración de un ciclo de desarrollo (Time-boxing)

Una estrategia muy útil en los ciclos de desarrollo consiste en limitarlo a un marco temporal, esto es, un lapso rígidamente fijo, digamos cuatro semanas. Todo el trabajo ha de concluirse en ese lapso. Un periodo entre dos semanas y dos meses suele ser conveniente. En un periodo menor sería muy difícil terminar las actividades; en un periodo mayor la complejidad se torna abrumadora y la retroalimentación se retrasa (figura 2.4).

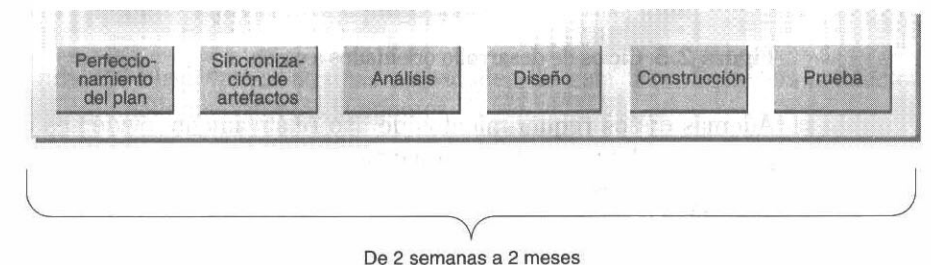


Figura 2.4 Fijación de la duración de un ciclo de desarrollo.

Para tener éxito con un programa de duración fija es necesario escoger los requerimientos con mucho cuidado y asignarle la selección al equipo del desarrollo: son ellos los responsables de cumplir con el plazo establecido.

2.4.2 Casos de uso y los ciclos iterativos del desarrollo

Un caso de uso es una descripción narrativa de un proceso de dominio; por ejemplo, *Obtener libros prestados en una biblioteca*.

Los ciclos iterativos de desarrollo se organizan a partir de los requerimientos del caso de uso.

Dicho de otra manera, se asigna un ciclo de desarrollo para implementar uno o más casos de uso o bien sus versiones simplificadas (por cierto muy comunes cuando el caso completo sería demasiado complejo de abordar en un ciclo) (figura 2.5).

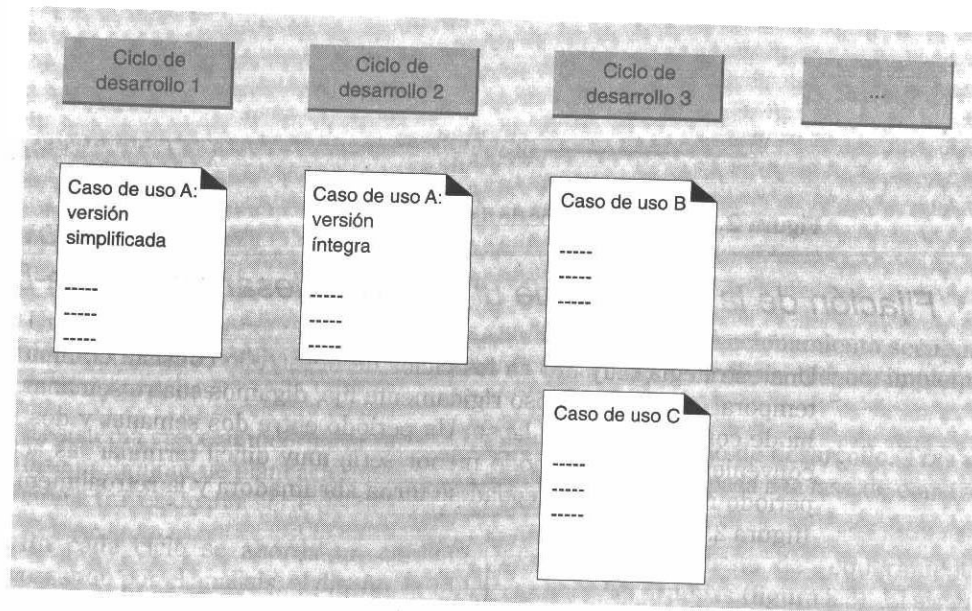


Figura 2.5 Ciclos de desarrollo orientados a casos.

Además de los requerimientos de uso relativamente evidentes que es preciso tener presentes, algunos ciclos —especialmente los primeros— han de centrarse en requerimientos poco evidentes; por ejemplo, la creación de servicios de soporte (persistencia, seguridad y otros).

2.4.3 Clasificación de los casos de uso

Deberían clasificarse los casos de uso, y los que ocupen los niveles más altos habrían de abordarse en los ciclos iniciales de desarrollo. La estrategia general consiste en seleccionar los casos que influyen profundamente en la arquitectura básica, dando soporte al dominio y a las capas de servicios de alto nivel o los que presentan el máximo riesgo.

2.5 La fase de la planeación y de la elaboración

Esta fase del proyecto incluye la concepción inicial, la investigación de alternativas, la planeación, la especificación de requerimientos y otras actividades.

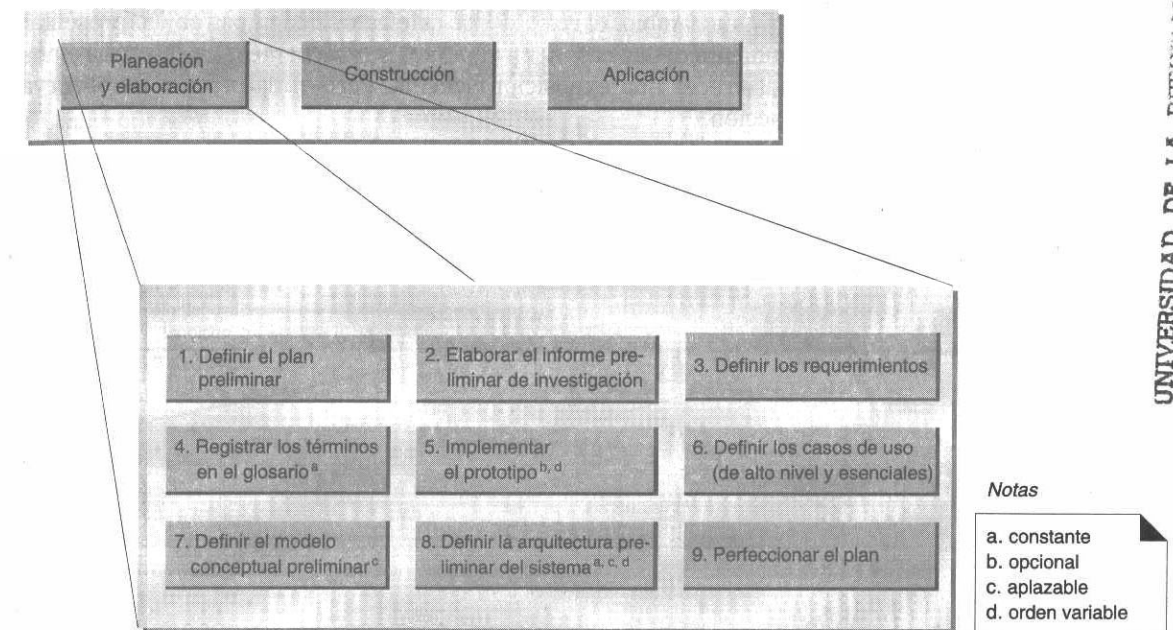


Figura 2.6 Ejemplo de actividades de la fase de planeación y elaboración.

En la figura 2.6 observamos algunas actividades de esta fase. Entre los artefactos generados aquí podemos citar los siguientes:

- **Plan:** programa, recursos, presupuesto, etcétera.
- **Informe preliminar de investigación:** motivos, alternativas, necesidades de la empresa.
- **Especificación de requerimientos:** declaración de los requerimientos.
- **Glosario:** diccionario (nombres de conceptos, por ejemplo) y toda información afín, como las restricciones y las reglas.

- **Prototipo:** sistema de prototipos cuyo fin es facilitar la comprensión del problema, los problemas de alto riesgo y los requerimientos.
- **Casos de uso:** descripciones narrativas de los procesos de dominio.
- **Diagramas de casos de uso:** descripción gráfica de todos los casos y de sus relaciones.
- **Bosquejo del modelo conceptual:** modelo conceptual preliminar cuya finalidad es facilitar el conocimiento del vocabulario del dominio, especialmente en su relación con los casos de uso y con las especificaciones de los requerimientos.

2.5.1 Orden de creación de los artefactos o elementos del desarrollo

Aunque la guía que se muestra en la figura 2.6 puede sugerir un orden lineal en la creación de los artefactos, no es estrictamente así. Podemos preparar en paralelo algunos de ellos, por ejemplo. Esto se aplica especialmente al modelo conceptual, al glosario, a los casos de uso y a los diagramas de los casos. Los casos de uso que son sometidos a examen y, en cambio, el resto de los artefactos tienen por objeto presentar la información proveniente de los casos. En capítulos subsecuentes, los describiremos en orden lineal para ofrecer una exposición sencilla. Pero en la práctica se observa mucha mayor interacción.

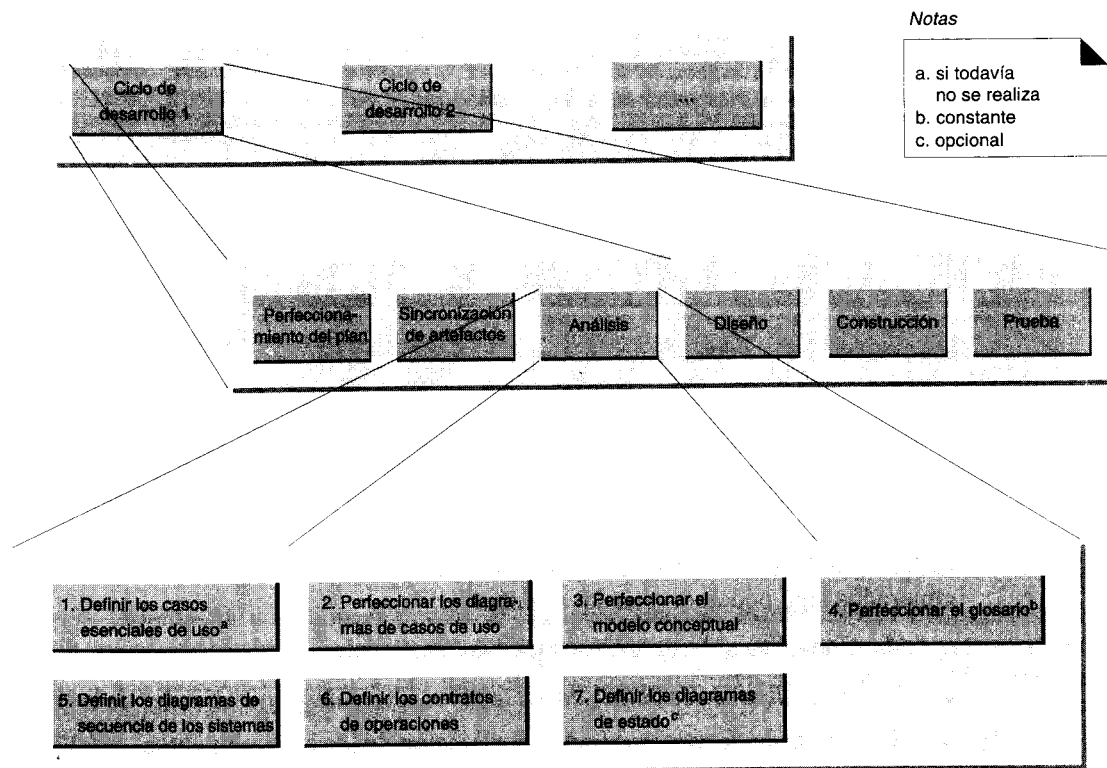


Figura 2.7 Ejemplo de las actividades en la fase de análisis.

2.6

La fase de construcción: ciclos del desarrollo

La fase de construcción de un proyecto requiere varios ciclos de desarrollo (probablemente con plazos fijos) a lo largo de los cuales se extiende el sistema. El objetivo final es obtener un sistema funcional de software que atienda debidamente los requerimientos.

En un ciclo individual de desarrollo, los principales pasos se analizan y diseñan, como se señala en las figuras 2.7 y 2.8. Los detalles de los pasos se comentan de manera pormenorizada en los siguientes capítulos y en el capítulo 37, donde examinaremos otros aspectos del proceso.

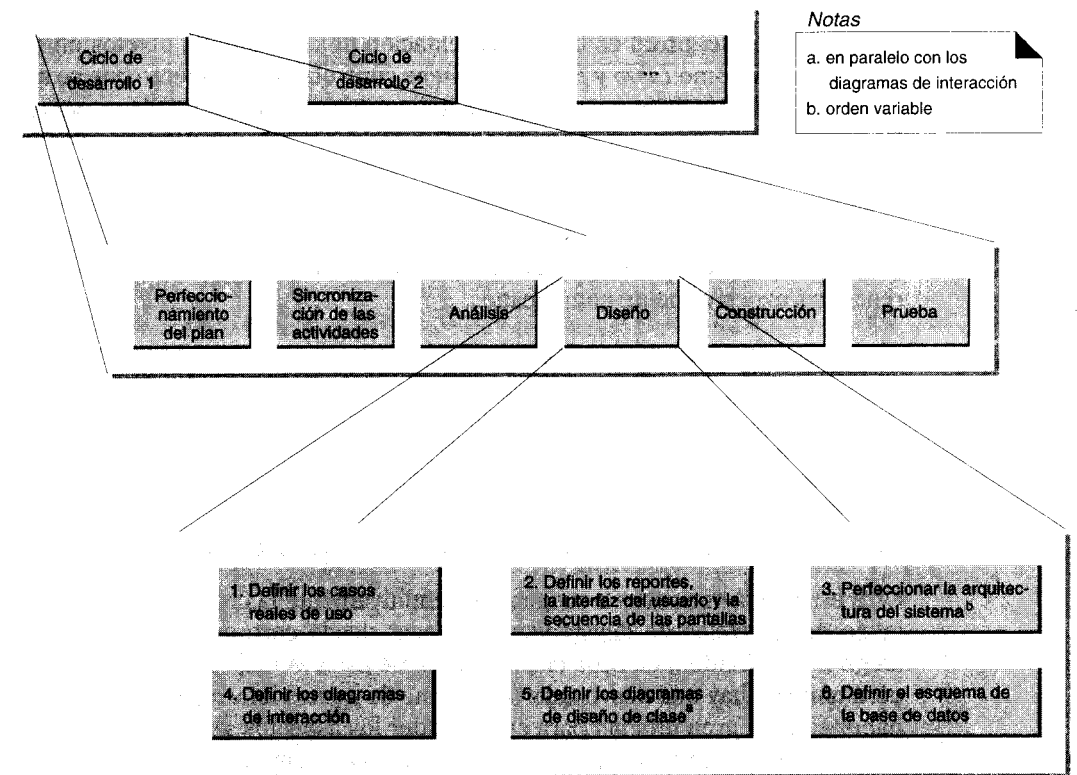


Figura 2.8 Ejemplo de las actividades de la fase de diseño.

2.6.1

Orden de creación de los artefactos en el ciclo de desarrollo

Como en el caso de los artefactos en la fase de requerimientos, el orden lineal deducible de la figura 2.7 no es el que se sigue rigurosamente. Algunos artefactos pueden elaborarse en paralelo; por ejemplo:

- Crear en paralelo un modelo conceptual y un glosario.
- Crear en paralelo los diagramas de interacción y los de diseño de clases.

2.7 Decidir cuándo crear artefactos

En la fase inicial de planeación y elaboración pueden crearse ciertos artefactos, entre ellos un modelo conceptual preliminar (un modelo de conceptos del mundo real) y casos expandidos de uso (descripciones narrativas detalladas de procesos). En la presente sección se examina el momento de su creación.

2.7.1 Cuándo crear el modelo conceptual

El **modelo conceptual** es una representación de conceptos u objetos en el dominio del problema, como *Libro* y *Biblioteca*. Debe controlarse el esfuerzo que se aplique a la producción del modelo conceptual preliminar durante la fase de planeación y elaboración. La meta es lograr un conocimiento básico del vocabulario y de los conceptos que se incluyen en los requerimientos. Por ello, no hace falta una investigación exhaustiva, pues se correría el riesgo de saturar demasiado la investigación desde el principio: exceso de complejidad. En los dominios de problemas amplios —por ejemplo, un sistema de reservaciones para las líneas aéreas—, un modelo conceptual muy completo resultaría excesivamente complicado.

La estrategia intermedia que recomendamos es generar rápidamente un modelo conceptual que se centre en identificar los conceptos obvios expresados en los requerimientos y posponer para más tarde una investigación con detenimiento. Más adelante, en cada ciclo de desarrollo, iremos refinando el modelo conceptual y ampliando los requerimientos referentes al ciclo.

Otra estrategia consiste en suspender definitivamente la creación del modelo hasta el inicio de los ciclos de desarrollo; comenzar desde cero en el ciclo de desarrollo y luego ir ampliándolo en cada ciclo. Se obtiene así la ventaja de aplazar la complejidad, pero también hay una desventaja: se dispone de menos información inicial, que pudiera haber sido de gran utilidad para comprender los aspectos generales, para entender el glosario, al realizar un examen meticuloso y al efectuar estimaciones. En el estudio de casos importantes se adopta este enfoque de posposición en la creación del modelo conceptual, no porque sea nuestro favorito sino porque favorece la explicación gradual de cómo producirlos.

2.7.2 Cuándo crear los casos expandidos de uso

Los **casos de uso de alto nivel** son muy breves, generalmente descripciones de un proceso en dos o tres oraciones. Los **casos expandidos de uso** son descripciones extensas que pueden contener cientos de oraciones con las cuales se realiza la descripción.

Durante la fase de planeación y elaboración, le aconsejamos crear todos los casos de uso de alto nivel, pero escribir sólo los más importantes en un formato expandido (largo), posponiendo el resto hasta el ciclo de desarrollo en que se estudian.

Igual que con el modelo conceptual, la adquisición temprana de información no está exenta de desventajas, pues puede originar una enorme complejidad. Investigar y escribir detalladamente en la fase de planeación y elaboración los casos de uso expandidos ofrece la ventaja de suministrar más información; ésta puede facilitar la comprensión, la administración del riesgo, el examen meticuloso y las estimaciones. Pero también ofrece la desventaja de una excesiva complejidad inicial: la investigación producirá miles de detalles nimios. Más aún, los casos expandidos tal vez no sean muy confiables porque la información es incompleta o errónea y porque los requerimientos pueden cambiar constantemente.

Por tanto, la estrategia intermedia que recomendamos es investigar detenidamente, durante la fase de planeación y elaboración, sólo los casos de uso más importantes.

DEFINICIÓN DE MODELOS Y ARTEFACTOS

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

Objetivos

- Definir los modelos de análisis y de diseño.
- Explicar con ejemplos las dependencias entre los artefactos de análisis y diseño.

3.1 Introducción

En este capítulo se exponen ejemplos de modelos en el análisis y diseño orientados a objetos, así como de la relación de subordinación entre los artefactos en los modelos.

El propósito es ofrecer al lector un panorama general; en capítulos posteriores profundizaremos en los detalles de los modelos y artefactos.

3.2 Sistemas de construcción de modelos

Un sistema (tanto en el mundo real como en el mundo del software) suele ser extremadamente intrincado; por ello es necesario dividir el sistema en partes o fragmentos si queremos entender y administrar su complejidad. Estas partes podemos representarlas como **modelos** que describan y abstraigan sus aspectos esenciales [Rumbaugh97].

Por tanto, un paso útil en la construcción de un sistema de software es el de crear modelos que organicen y comuniquen los detalles importantes del problema de la vida

real con que se relacionan y del sistema a construir. Los modelos deben contener elementos cohesivos y estrechamente interconexos.

UML son las siglas de Unified *Modeling* Language (Lenguaje Unificado de Construcción de Modelos), porque su finalidad es describir modelos de sistemas (del mundo real y del mundo del software), basados en los conceptos de objetos.

Los modelos se componen de otros modelos o **artefectos**, de diagramas y documentos que describen cosas. El UML especifica varios diagramas, entre ellos los diagramas de casos de uso y los diagramas de interacción, que son los artefactos concretos a partir de los cuales creamos los modelos. Estos se visualizan por medio de las **vistas** —proyecciones visuales del modelo—; los diagramas de UML, entre ellos los de interacción, abarcan las vistas de un modelo.

Si queremos caracterizar los modelos, podemos poner de manifiesto la información **estática** o **dinámica** de un sistema. Un **modelo estático** describe las propiedades estructurales del sistema; en cambio, un **modelo dinámico** describe las propiedades de comportamiento de un sistema.

3.3 Modelos muestra

El UML es un lenguaje flexible de modelado que permite definir modelos arbitrarios. En esta sección presentaremos ejemplos de modelos de análisis y de diseño que expliquen la pertenencia de artefactos concretos a los modelos.

Se trata de modelos *muestra*; no se pretende en absoluto que sean definitivos.

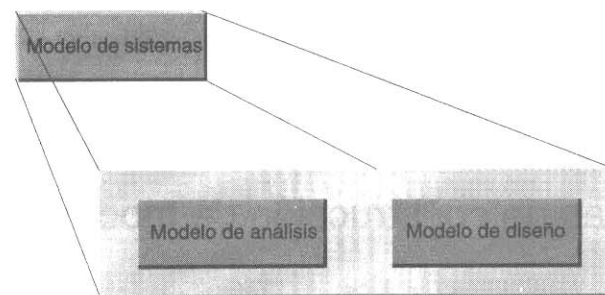


Figura 3.1 El modelo de sistemas.

3.3.1 El modelo del sistema

En este ejemplo el modelo global del sistema (figura 3.1) está constituido por el:

- **Modelo de análisis:** el que se relaciona con una investigación del dominio y del ámbito del problema, pero no con la solución.
- **Modelo de diseño:** el que se relaciona con la solución lógica.

En capítulos subsecuentes se estudian estos modelos (figura 3.1) con mayor detalle.

3.4

Relación entre los artefactos

Sin importar cómo los artefactos se organicen para construir modelos, se dan dependencias muy importantes entre ellos. Por ejemplo, un diagrama de casos de uso que muestre todos los casos depende de las definiciones de los casos. Conviene entender la dependencia y la influencia entre los artefactos para poder efectuar comprobaciones de consistencia y de rastreabilidad y para poder utilizar eficazmente los artefactos dependientes como punto de partida para crear otros.¹

Por ejemplo, en la figura 3.2 se observan gráficamente las dependencias entre artefactos generados durante la fase de planeación y elaboración. En los siguientes capítulos trataremos más a fondo del significado de los artefactos y sus dependencias.

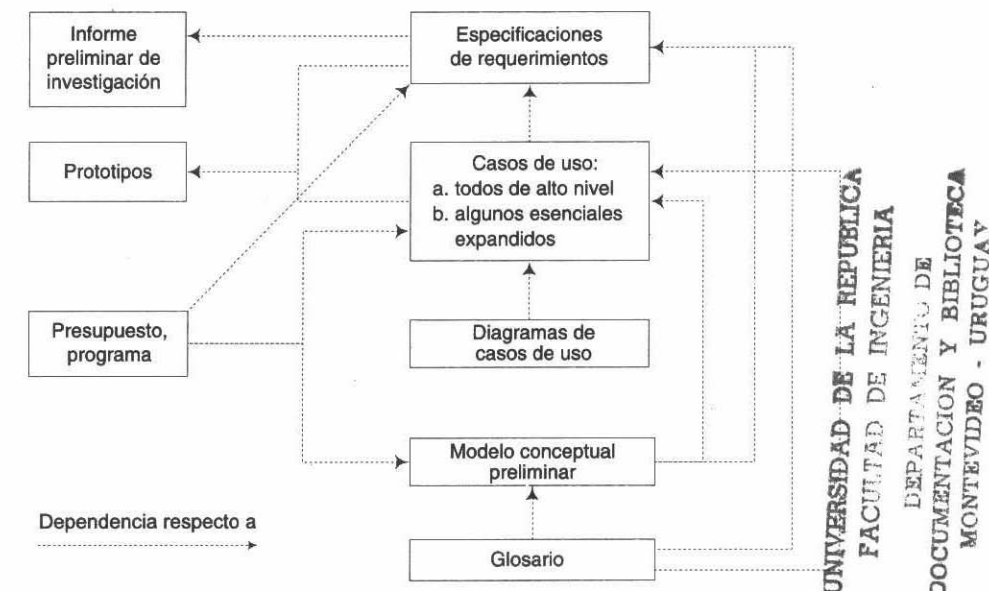


Figura 3.2 Influencia de los artefactos en la fase de planeación y de elaboración.

¹ Si se crea un artefacto que no tenga otros dependientes y si no se usa como entrada de otra cosa, habrá que poner en tela de juicio su valor y el tiempo que se dedicó a su creación.

PARTE II FASE DE
PLANEACIÓN Y
DE ELABORACIÓN

CASO DE ESTUDIO: EL PUNTO DE VENTA

Objetivos

- Definir el caso de estudio que se emplea en el libro.

4.1 El sistema del punto de venta

Nuestro principal caso de estudio es el sistema de una terminal (POST) de punto de venta.¹ Esta terminal es un sistema computarizado con el que se registran las ventas y se realizan los pagos; normalmente se utiliza en las tiendas al detalle. Abarca componentes de hardware (una computadora y un lector de código de barras) y software para correr el sistema.

Suponga que se nos ha pedido crear un programa para una terminal de punto de venta. Con una estrategia de desarrollo de incremento iterativo, vamos a realizar las fases de requerimientos, análisis y diseño orientados a objetos e implementación.

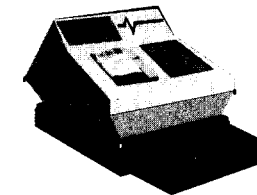


Figura 4.1 Terminal de punto de venta.

¹ Un problema también examinado en [Coad95], aunque este trabajo se llevó a cabo de manera independiente y mucho antes que el otro.

¿Por qué escogimos este problema? Por ser representativo de muchos sistemas de información y porque toca problemas comunes que puede encontrar un desarrollador. Ahondaremos lo suficiente en el análisis y en el diseño, para que el lector vea en forma detallada cómo se abordan estos problemas y pueda aplicar el método a otros proyectos.

4.2 Capas arquitectónicas y el énfasis en el caso de estudio

Un sistema típico de información que incluya una interfaz gráfica del usuario y acceso a la base de datos suele presentar un diseño arquitectónico de varios niveles o capas (figura 4.2) como las siguientes:

- **Presentación:** interfaz gráfica; ventanas.
- **Lógica de aplicación - Objetos del dominio del problema:** objetos que representan conceptos del dominio (los objetos de ventas, por ejemplo) que cumplen con los requisitos de aplicación.
- **Lógica de aplicación - Objetos de servicio:** objetos de dominio no relacionados con el problema que prestan servicios de soporte; por ejemplo, interfaz con una base de datos.
- **Almacenamiento:** un mecanismo persistente de almacenamiento; por ejemplo una base de datos relacional u orientada a objetos.

El análisis y diseño orientados a objetos generalmente son más útiles para modelar los niveles lógicos de la aplicación.

Estudiaremos en el capítulo 22 el tema de una arquitectura de capas (o niveles).

El caso de estudio del punto de venta destaca principalmente los objetos del dominio del problema, asignándoles responsabilidades para cumplir con los requisitos de la aplicación. En el capítulo 38 nos serviremos del diseño orientado a objetos para crear un conjunto de objetos del nivel de servicio para conectarnos con una base de datos.

En este método de diseño, la capa de presentación tiene muy poca responsabilidad; se dice que es *delgada*. Las ventanas *no* contienen un código que se encargue de la lógica o procesamiento de la aplicación. Por el contrario, las solicitudes de tarea se envían al dominio del problema y a las capas de servicio.

4.3 Nuestra estrategia: aprendizaje y desarrollo iterativos

Este libro está organizado para seguir una estrategia de desarrollo iterativo. El análisis y el diseño orientados a objetos se aplican al sistema del punto de venta en dos ciclos de desarrollo iterativo en que el primer ciclo se destina a una simple aplicación de las funciones básicas. El segundo amplía la funcionalidad del sistema (véase la figura 4.3).

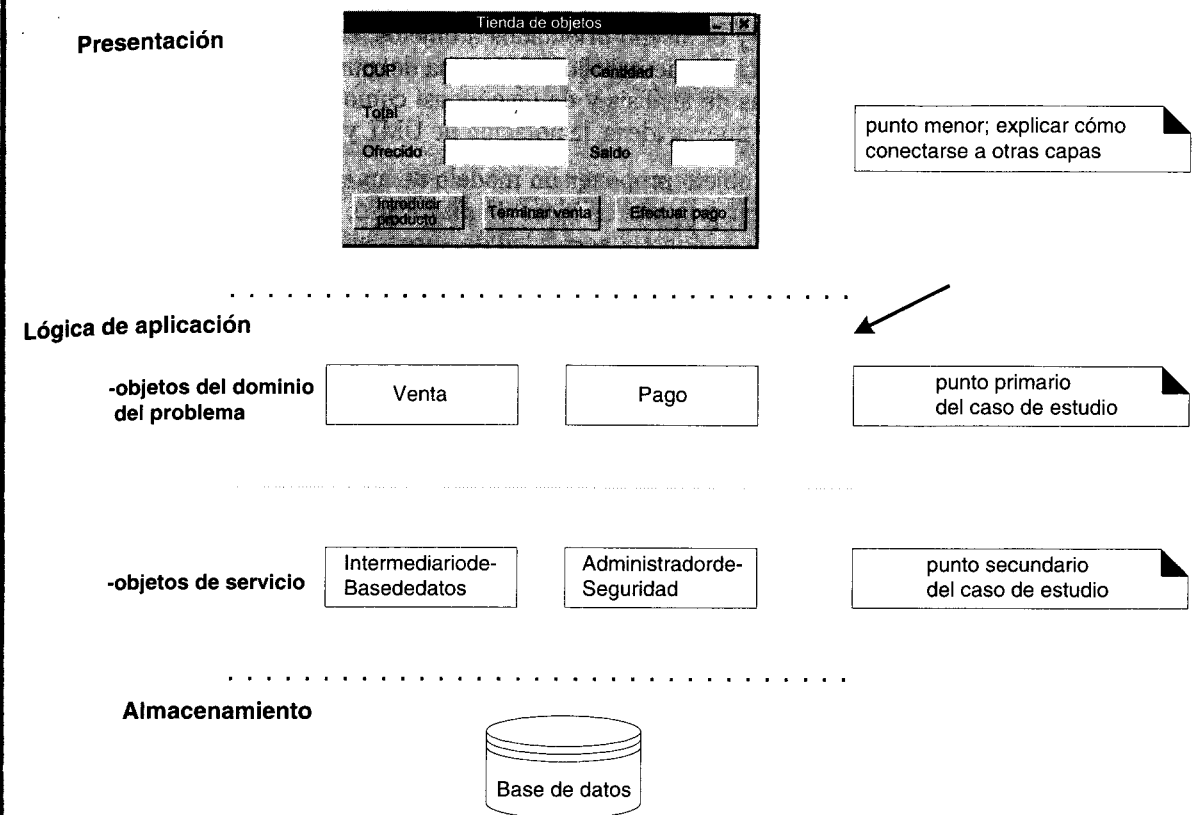


Figura 4.2 Capas de un sistema ordinario de información orientado a objetos.

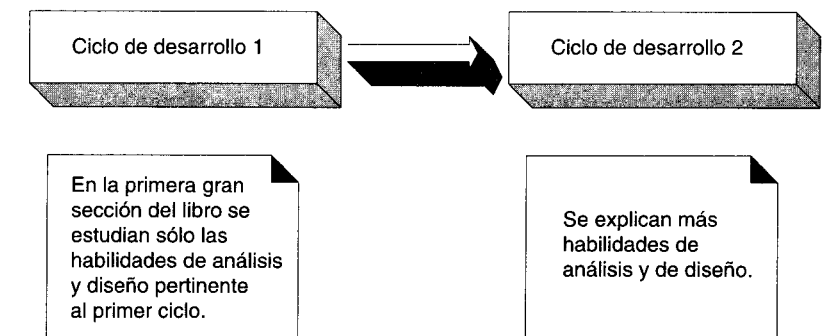


Figura 4.3 Ruta de aprendizaje que siguen los ciclos de desarrollo.

Además del desarrollo iterativo, también se expone de manera iterativa la *presentación* de los temas del análisis y el diseño orientados a objetos, la notación de UML y los patrones. En el primer ciclo de desarrollo del sistema del punto de venta, se describe un grupo básico de temas de análisis y de diseño, así como la notación. El segundo ciclo se expande y ofrece nuevas ideas, la notación de UML y los patrones.

Esta estrategia tiene por objeto proponer un modelo de aprendizaje “justo a tiempo”. El modelo procura explicar primero las ideas de mayor uso, lo más cerca posible del momento en que el lector perciba la necesidad de aprenderlos para poder continuar. Al principio, el libro se centra en las ideas y habilidades fundamentales, sin saturarlo con nueva información que no pueda aplicar de inmediato. Después se explican las habilidades e ideas que se utilizan más raramente.

CONOCIMIENTO DE LOS REQUERIMIENTOS

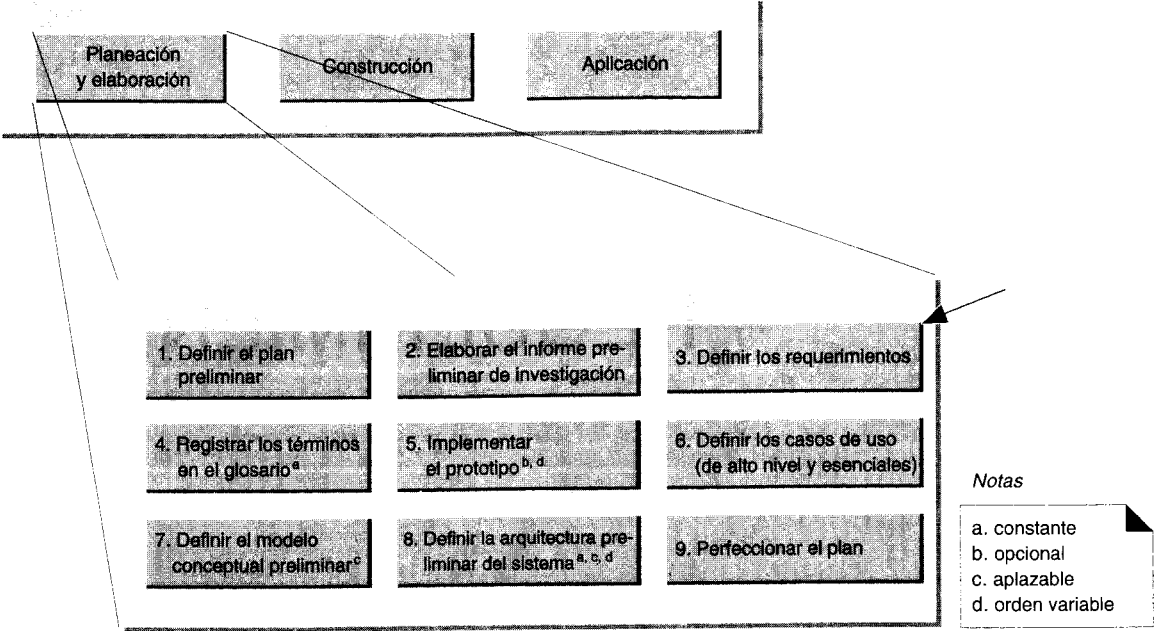
Objetivos

- Crear los artefactos de la fase de requerimientos; por ejemplo, las especificaciones de funciones.
- Identificar y clasificar las funciones del sistema.
- Identificar y clasificar los atributos del sistema y relacionarlos con las funciones.

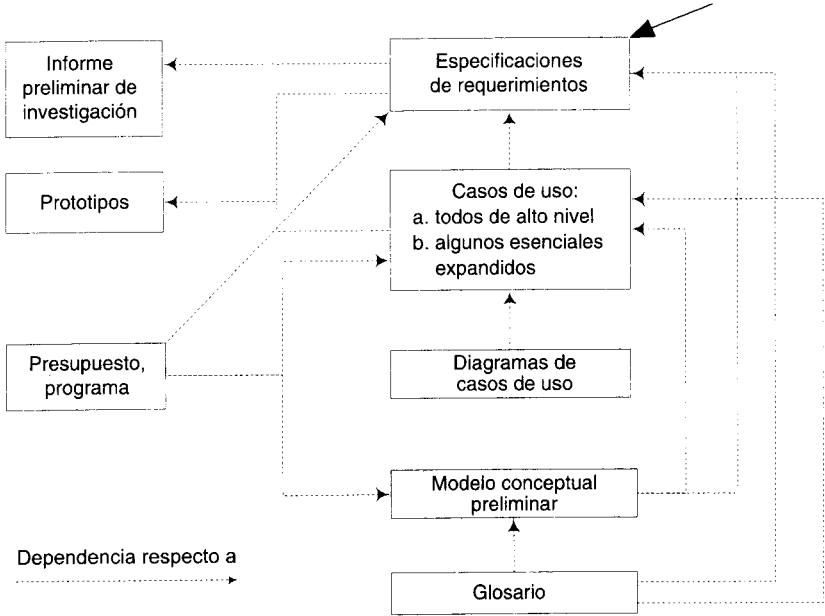
5.1 Introducción

Un proyecto no puede ser exitoso sin una especificación correcta y exhaustiva de los requerimientos. Para ello se necesitan muchas habilidades; un examen riguroso de ellas rebasa el ámbito de este libro, pues nuestro objetivo es que el lector domine el análisis y el diseño orientados a objetos. Pero se ofrece una introducción a los requerimientos de la aplicación punto de venta, porque en la práctica es un paso decisivo, y se mencionan otros artefactos relacionados con la fase de requerimientos. No dudamos en recomendar *Exploring Requirements: Quality Before Design* [GW89], obra que se centra en las habilidades necesarias para dilucidar los requerimientos importantes.

Este capítulo se propone lograr que el lector pueda expresar los requerimientos, no en convertirlo en un experto en el dominio de los sistemas de terminales para tiendas y puntos de venta. Por eso, la lista de las funciones y atributos del sistema no es exhaustiva, sino representativa.



Actividades de la fase de planeación y elaboración.



Dependencias de los artefactos respecto a la fase de planeación y elaboración.

Ninguno de los artefactos que se describen en la presente sección son propios del lenguaje UML; se trata simplemente de documentos comunes de la fase de requerimientos.

5.2 Los requerimientos

Los **requerimientos** son una descripción de las necesidades o deseos de un producto. La meta primaria de la fase de requerimientos es identificar y documentar lo que en realidad se necesita, en una forma que claramente se lo comunique al cliente y a los miembros del equipo de desarrollo. El reto consiste en definirlos de manera inequívoca, de modo que se detecten los riesgos y no se presenten sorpresas al momento de entregar el producto.

Se recomiendan los siguientes artefactos en la fase de requerimientos:

- panorama general
- clientes
- metas
- funciones del sistema
- atributos del sistema

Otros documentos pertinentes, que por cierto no examinaremos en el libro, se mencionan al final del capítulo.

5.2.1 Integración de las piezas del rompecabezas

En nuestro caso de estudio, la definición de los requerimientos aparece muy clara y tajante: la realidad dista mucho de serlo. Por lo regular hay que reunir y asimilar muchos estudios y documentos electrónicos, analizar los resultados de las entrevistas, celebrar reuniones para definir los requerimientos en grupo, etcétera.

5.3 Presentación general

Este proyecto tiene por objeto crear un sistema de terminal para el punto de venta que se utilizará en las ventas al menudeo.

5.4 Clientes

ObjectStore, Inc., detallista multinacional de objetos.

5.5 Metas

En términos generales, la meta es una mayor automatización del pago en las cajas registradoras, dar soporte a servicios más rápidos, más baratos y mejores y a los procesos de negocios. Más concretamente, la meta incluye:

- Pago rápido de los clientes.
- Análisis rápido y exacto de las ventas.
- Control automático del inventario.

5.6 Funciones del sistema

Las **funciones del sistema** son lo que éste habrá de *hacer*, por ejemplo autorizar los pagos a crédito. Hay que identificarlas y listarlas en grupos cohesivos y lógicos.

Con el objeto de verificar que algún X es de verdad una función del sistema, la siguiente oración deberá tener sentido:

El sistema deberá hacer <X>.

Por ejemplo: *El sistema deberá autorizar los pagos a crédito.*

En cambio, los **atributos del sistema** son cualidades no funcionales —entre ellas la facilidad de uso— que a menudo se confunden con las funciones. Nótese que “facilidad de uso” no encaja en la oración de verificación: *El sistema deberá hacer la facilidad de uso*. Los atributos no deben formar parte del documento de las especificaciones funcionales del sistema, sino de un documento independiente que especifica sus atributos.

5.6.1 Categorías de las funciones

Las funciones, como *autorizar pagos a crédito*, han de clasificarse a fin de establecer prioridades entre ellas e identificar las que de lo contrario pasarían inadvertidas (pero que consumen tiempo y otros recursos). Las categorías son:

Categoría de la función	Significado
Evidente	Debe realizarse, y el usuario debería saber que se ha realizado.
Ocultas	Debe realizarse, aunque no es visible para los usuarios. Esto se aplica a muchos servicios técnicos subyacentes, como <i>guardar información en un mecanismo persistente de almacenamiento</i> . Las funciones ocultas a menudo se omiten (erróneamente) durante el proceso de obtención de los requerimientos.
Superflua	Opcionales; su inclusión no repercute significativamente en el costo ni en otras funciones.

5.6.2 Funciones básicas

Las siguientes funciones del sistema en la aplicación de la terminal del punto de venta son una muestra representativa; no pretenden en absoluto ser exhaustivas. Nuestro objetivo es entender los detalles del análisis y del diseño, no el funcionamiento de una tienda.

Ref #	Función	Categoría
R1.1	Registra la venta en proceso (actual): los productos comprados.	evidente
R1.2	Calcula el total de la venta actual; se incluyen el impuesto y los cálculos de cupón.	evidente
R1.3	Captura la información sobre el objeto comprado usando su código de barras y un lector o usando una captura manual de un código del producto; por ejemplo, un código universal de producto (UPC).	evidente
R1.4	Reduce las cantidades del inventario cuando se realiza una venta.	oculta
R1.5	Se registran las ventas efectuadas.	oculta
R1.6	El cajero debe introducir una identificación y una contraseña para poder utilizar el sistema.	evidente
R1.7	Ofrece un mecanismo de almacenamiento persistente.	oculta

Ref #	Función	Categoría
R1.8	Ofrece mecanismos de comunicación entre los procesos y entre los sistemas.	oculta
R1.9	Muestra la descripción y el precio del producto registrado.	evidente

5.6.3 Funciones de pago

Ref #	Función	Categoría
R2.1	Maneja los pagos en efectivo, capturando la cantidad ofrecida y calculando el saldo deudor.	evidente
R2.2	Maneja los pagos a crédito, capturando la información crediticia a partir de una lectora de tarjetas o mediante captura manual, y autorizando los pagos con el servicio de autorización (externa) de créditos de la tienda a través de una conexión por módem.	evidente
R2.3	Maneja los pagos con cheque, capturando la licencia de conducir mediante captura manual, y autorizando los pagos con el servicio de autorización (externa) de cheques de la tienda a través de la conexión por módem.	evidente
R2.4	Registra los pagos en el sistema de cuentas por cobrar, pues el servicio de autorización de crédito debe a la tienda el monto del pago.	oculta

5.7 Atributos del sistema

Los atributos del sistema son sus características o dimensiones; no son funciones. Por ejemplo:

- facilidad de uso
- tolerancia a las fallas
- tiempo de respuesta
- metáfora de interfaz
- costo al detalle
- plataformas

Los atributos del sistema pueden abarcar todas las funciones (por ejemplo, la plataforma del sistema operativo) o ser específicos de una función o grupo de funciones.

Los atributos tienen un posible conjunto de **detalles de atributos**, los cuales tienden a ser valores discretos, confusos o simbólicos; por ejemplo:

- tiempo de respuesta = (psicológicamente correcto)
- metáfora de interfaz = (gráfico, colorido, basado en formas)

Algunos atributos del sistema también pueden tener **restricciones de frontera del atributo**, que son condiciones obligatorias de frontera, generalmente en un rango numérico de los valores de un atributo; por ejemplo:

- tiempo de respuesta = (cinco segundos como máximo)

He aquí algunos ejemplos más:

Atributo	Detalles y restricciones de frontera
tiempo de respuesta	(<i>restricción de frontera</i>) Cuando se registre un producto vendido, la descripción y el precio aparecerán en cinco segundos.
metáfora de interfaz	(<i>detalle</i>) Ventanas orientadas a la metáfora de una forma y cuadros de diálogo. (<i>detalle</i>) maximiza una navegación fácil con teclado y no con apuntadores.
tolerancia a fallas	(<i>restricción de frontera</i>) debe registrar los pagos a crédito autorizados que se hagan a las cuentas por cobrar en un plazo de 24 horas, aun cuando se produzcan fallas de energía o del equipo.
plataformas del sistema operativo	(<i>detalle</i>) Microsoft Windows 95 y NT.

5.7.1 Atributos del sistema en las especificaciones de funciones

Conviene describir todos los atributos del sistema que se relacionen claramente con las funciones *dentro* de la lista en que se especifican estas últimas. Además, los detalles de los atributos y las restricciones de frontera pueden catalogarse como *obligatorios* u *opcionales*.¹

¹ Una restricción de frontera suele ser *obligatoria*, pues de lo contrario significaría que no era sólida.

Ref #	Función	Cat.	Atributo	Detalles y restricciones	Cat.
R1.9	Mostrar la descripción y el precio del producto registrado.	evidente	tiempo de respuesta	5 segundos como máximo	obligatorio
			metáfora de interfaz	pantallas basadas en formas colorido	obligatorio opcional
R2.4	Registrar los pagos a crédito en el sistema de cuentas por cobrar, pues el servicio de autorización de crédito debe a la tienda el importe del pago.	oculto	tolerancia a fallas	debe registrar en las cuentas por cobrar en un plazo de 24 horas, aun cuando se produzcan fallas de energía o del equipo	obligatorio
			tiempo de respuesta	10 segundos como máximo	obligatorio

5.8 Otros artefactos en la fase de los requerimientos

En este libro se da una introducción muy sucinta a los requerimientos; es un tema que bien podría abarcar libros enteros. Las funciones y los atributos del sistema son los documentos mínimos de los requerimientos, de modo que se necesitan otros artefactos importantes para atenuar el riesgo y entender el problema, a saber:

- *Requerimientos y equipos de enlace:* lista de los que deberían participar en la especificación de las funciones y atributos del sistema, en la realización de entrevistas, de pruebas, de negociaciones y de otras actividades.
- *Grupos afectados:* los que reciben el impacto del desarrollo o aplicación del sistema.
- *Suposiciones:* las cosas cuya verdad se supone.
- *Riesgos:* las cosas que pueden ocasionar el fracaso o retraso.
- *Dependencias:* otras personas, sistemas y productos de los cuales no puede prescindir el proyecto para su terminación.
- *Glosario:* definición de los términos pertinentes; tema que se estudiará en capítulos subsecuentes.
- *Casos de uso:* descripciones narrativas de los procesos del dominio; tema que se verá en capítulos posteriores.
- *Modelo conceptual preliminar:* modelo de conceptos importantes y de sus relaciones; tema que se tratará en capítulos posteriores.

CASOS DE USO:
DESCRIPCIÓN DE PROCESOS

Objetivos

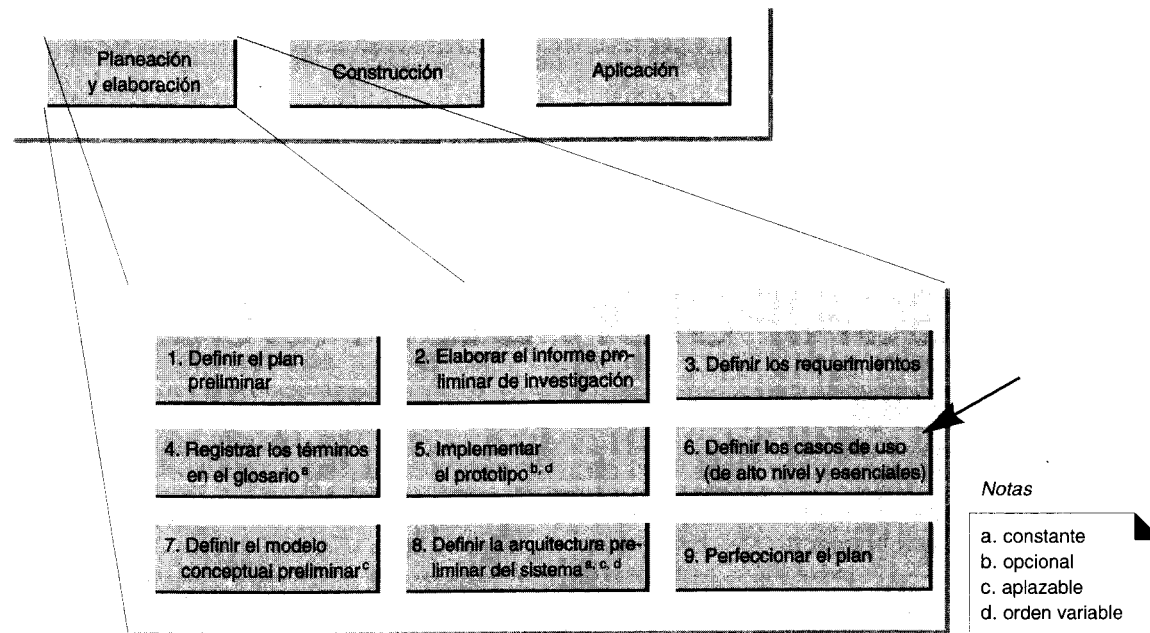
- Identificar y escribir casos de uso.
- Diseñar diagramas de casos de uso.
- Contrastar los casos de uso de alto nivel con los expandidos.
- Contrastar los casos de uso esenciales con los reales.

6.1 Introducción

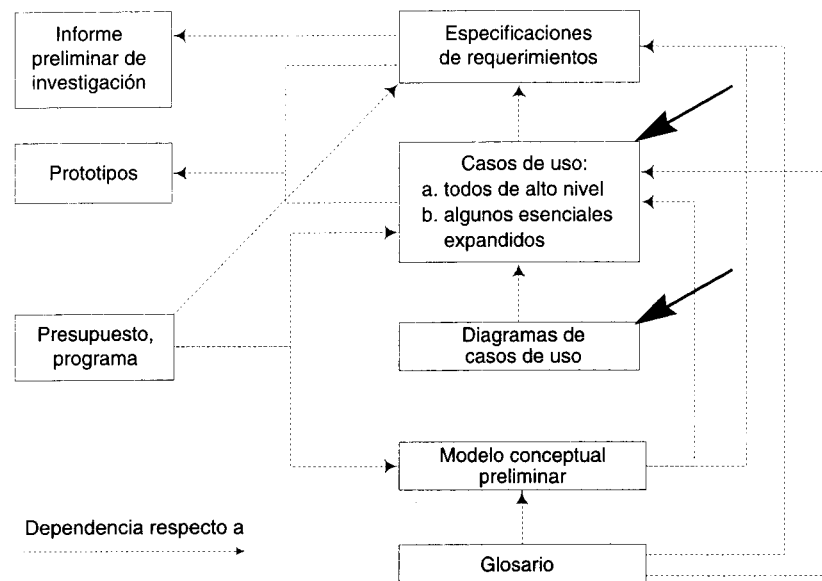
Una técnica excelente que permite mejorar la comprensión de los requerimientos es la creación de casos de uso, es decir, descripciones narrativas de los procesos del dominio. En este capítulo se exponen los conceptos básicos de esta técnica y se dan ejemplos para aplicarlos a la terminal de punto de venta.

En el siguiente capítulo clasificaremos los casos de uso y escogeremos los que utilizaremos en el primer ciclo de desarrollo.

El UML incluye formalmente el concepto de casos de uso y sus diagramas de uso.



Actividades de la fase de planeación y elaboración.



Dependencias de los artefactos respecto a la fase de planeación y elaboración.

6.2 Actividades y dependencias

Los casos de uso requieren tener al menos un conocimiento parcial de los requerimientos del sistema, en teoría expresados en el documento donde se especifican.

6.3 Casos de uso

El **caso de uso** es un documento narrativo que describe la secuencia de eventos de un actor (agente externo) que utiliza un sistema para completar un proceso [Jacobson92]. Los casos de uso son historias o casos de utilización de un sistema; no son exactamente los requerimientos ni las especificaciones funcionales, sino que ejemplifican e incluyen tácitamente los requerimientos en las historias que narran.

Comprar productos

Figura 6.1 Icono del lenguaje UML para un caso de uso.

6.3.1 Ejemplo de un caso de uso de alto nivel: comprar productos

El siguiente caso de uso de alto nivel describe clara y concisamente el proceso de comprar artículos en una tienda cuando se emplea una terminal en el punto de venta.

Caso de uso:	Comprar productos
Actores:	Cliente, Cajero.
Tipo:	Primario (que se explicará luego).
Descripción:	Un Cliente llega a la caja registradora con los artículos que comprará. El Cajero registra los artículos y cobra el importe. Al terminar la operación, el Cliente se marcha con los productos.

Los encabezados y la estructura de este caso de uso son representativos. Sin embargo, el UML no especifica un formato rígido; puede modificarse para atender las necesidades y ajustarse al espíritu de la documentación: ante todo, una comunicación clara.

Conviene comenzar con los casos de uso de alto nivel para lograr rápidamente entender los principales procesos globales.

6.3.2 Ejemplo de un caso expandido de uso:
comprar productos con efectivo

Un **caso expandido de uso** muestra más detalles que uno de alto nivel; este tipo de casos suelen ser útiles para alcanzar un conocimiento más profundo de los procesos y de los requerimientos. A menudo se llevan a cabo en un estilo “coloquial” entre los actores y el sistema [Wirfs-Brock93]. Damos en seguida un ejemplo de un caso expandido de *Comprar productos* que ha sido simplificado para manejar únicamente los pagos en efectivo y excluir la administración del inventario (lo hemos hecho para simplificar la explicación en este primer ejemplo).

Caso de uso:	Comprar productos en efectivo
Actores:	Cliente (iniciador), Cajero.
Propósito:	Capturar una venta y su pago en efectivo.
Resumen:	Un Cliente llega a la caja registradora con artículos que desea comprar. El Cajero registra los productos y recibe un pago en efectivo. Al terminar la operación, el Cliente se marcha con los productos comprados.
Tipo:	Primario y esencial.
Referencias cruzadas:	Funciones: R1.1, R1.2, R1.3, R1.7, R1.9, R2.1.

Curso normal de los eventos	
Acción del actor	Respuesta del sistema
1. Este caso de uso comienza cuando un Cliente llega a una caja de TPDV (Terminal Punto de Venta) con productos que desea comprar.	
2. El Cajero registra el identificador de cada producto. Si hay varios productos de una misma categoría, el Cajero también puede introducir la cantidad.	3. Determina el precio del producto e incorpora a la transacción actual la información correspondiente. Se presentan la descripción y el precio del producto actual.
4. Al terminar de introducir el producto, el Cajero indica a TPDV que se concluyó la captura del producto.	5. Calcula y presenta el total de la venta.
6. El Cajero le indica el total al Cliente.	

Curso normal de los eventos

Acción del actor	Respuesta del sistema
7. El Cliente efectúa un pago en efectivo —el “efectivo ofrecido”— posiblemente mayor que el total de la venta.	
8. El Cajero registra la cantidad de efectivo recibida.	9. Muestra al cliente la diferencia. Genera un recibo.
10. El Cajero deposita el efectivo recibido y extrae el cambio del pago. El Cajero da al Cliente el cambio y el recibo impreso.	11. Registra la venta concluida.
12. El Cliente se marcha con los artículos comprados.	

Cursos alternos

- Línea 2: introducción de identificador inválido. Indicar error.
- Línea 7: el cliente no tenía suficiente dinero. Cancelar la transacción de venta.

6.3.3 Explicación del formato expandido

La parte superior de la forma expandida es información muy sucinta.

Caso de uso:	Nombre del caso de uso
Actores:	Lista de actores (agentes externos), en la cual se indica quién inicia el caso de uso.
Propósito:	Intención del caso de uso.
Resumen:	Repetición del caso de uso de alto nivel o alguna síntesis similar.
Tipo:	1. Primario, secundario u opcional (a explicar). 2. Esencial o real (a explicar).
Referencias cruzadas:	Casos relacionados de uso y funciones también relacionadas del sistema.

La sección intermedia, *curso normal de los eventos*, es la parte medular del formato expandido; describe los detalles de la conversión interactiva entre los actores y el sistema. Un aspecto esencial de la sección es que explica la secuencia más común de los eventos: la historia normal de las actividades y la terminación exitosa de un proceso. *No* incluye situaciones alternas.

Curso normal de los eventos

Acción del actor	Respuesta del sistema
Acciones numeradas de los actores.	Descripciones numeradas de las respuestas del sistema.

La última sección, *curso alterno de los eventos*, describe importantes opciones o excepciones que pueden presentarse en relación con el curso normal. Si son complejas, podemos expandirlas y convertirlas en nuestros casos de uso.

Cursos alternos

- Alternativas que pueden ocurrir en el número de línea. Descripción de excepciones.

6.4 Actores

El actor es una entidad externa del sistema que de alguna manera participa en la historia del caso de uso. Por lo regular estimula el sistema con eventos de entrada o recibe algo de él. Los actores están representados por el papel que desempeñan en el caso: Cliente, Cajero u otro. Conviene escribir su nombre con mayúscula en la narrativa del caso para facilitar la identificación.

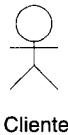


Figura 6.2 Icono del lenguaje UML que representa un actor de casos de uso.¹

¹ Aunque el icono estándar es una figura humana estilizada, hay quienes prefieren utilizar un icono con figura de computadora para designar los actores que son sistemas de cómputo y no seres humanos.

En un caso de uso hay un **actor iniciador** que produce la estimulación inicial y, posiblemente, otros **actores participantes**; tal vez convenga indicar quién es el iniciador.

Los actores suelen ser los papeles representados por seres humanos, pero pueden ser cualquier tipo de sistema, como un sistema computarizado externo de bancos. He aquí algunos tipos:

- papeles que desempeñan las personas
- sistemas de cómputo
- aparatos eléctricos o mecánicos

6.5 Un error común en los casos de uso

Un error común en la identificación de los casos de uso consiste en representar los pasos, las operaciones o las transacciones individuales como casos. Por ejemplo, en el dominio de la terminal del punto de venta, podemos definir (incorrectamente) un caso denominado “Imprimir el recibo”, cuando en realidad esta operación no es más que un paso de un proceso más amplio del caso *Comprar productos*.

Un caso de uso es una descripción de un proceso de principio a fin relativamente amplia, descripción que suele abarcar muchos pasos o transacciones; normalmente no es un paso ni una actividad individual del proceso.

Es posible dividir las actividades o parte del caso en subcasos (denominados **casos abstractos de uso**), incluso en pasos individuales; pero esto no es lo habitual y lo veremos en el capítulo 26.

6.6 Identificación de los casos de uso

Los siguientes pasos de la identificación de los casos de uso requieren una lluvia de ideas y revisar los documentos actuales sobre la especificación de los requerimientos.

Un método con que se identifican los casos de uso se basa en los actores.

1. Se identifican los actores relacionados con un sistema o empresa.
2. En cada actor, se identifican los procesos que inician o en que participan.

Un segundo método de identificación de los casos de uso se basa en eventos.

1. Se identifican los eventos externos a los que un sistema ha de responder.
2. Se relacionan los eventos con los actores y con los casos de uso.

En la aplicación del punto de venta, algunos actores posiblemente relevantes y los procesos que inician son:

Cajero	Registra Entrega efectivo
Cliente	Compra productos Paga los productos

6.7 Caso de uso y procesos del dominio

Un caso de uso describe un proceso, un proceso de negocios por ejemplo. Un **proceso** describe, de comienzo a fin, una secuencia de los eventos, de las acciones y las transacciones que se requieren para producir u obtener algo de valor para una empresa o actor.

A continuación se mencionan algunos procesos:

- Retira efectivo en un cajero automático
- Ordena un producto
- Registra los cursos que se imparten en una escuela
- Verifica la ortografía de un documento con un procesador de palabras
- Realiza una llamada telefónica

6.8 Casos de uso, funciones del sistema y rastreabilidad

Las funciones del sistema identificadas durante la especificación previa de requerimientos deben asignarse a los casos de uso. Además, debe ser posible verificar, mediante la sección *Referencias cruzadas*, que todas las funciones hayan sido asignadas. Con ello se logra un vínculo importante respecto a la rastreabilidad entre los artefactos. En definitiva, todas las funciones y casos de uso del sistema deberían poder rastrearse hasta la implementación y la aplicación de pruebas.

6.9 Diagramas de los casos de uso

En la figura 6.3 se muestra un diagrama de casos de uso para el sistema del punto de venta.

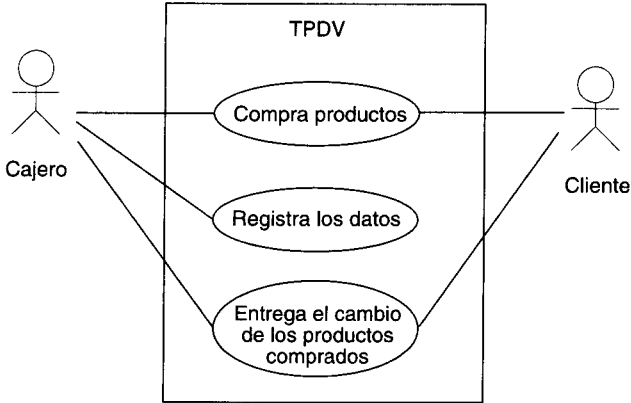


Figura 6.3 Diagrama parcial de casos de uso.

Un **diagrama de casos de uso** explica gráficamente un conjunto de casos de uso de un sistema, los actores y la relación entre éstos y los casos de uso. Estos últimos se muestran en óvalos y los actores son figuras estilizadas. Hay líneas de comunicaciones entre los casos y los actores; las flechas indican el flujo de la información o el estímulo.

El diagrama tiene por objeto ofrecer una clase de diagrama contextual que nos permite conocer rápidamente los actores externos de un sistema y las formas básicas en que lo utilizan.

6.10 Formatos de los casos de uso

En la práctica, los casos de uso pueden expresarse con diverso grado de detalle y de aceptación de las decisiones concernientes al diseño. En otras palabras, un mismo

caso de uso pueden escribirse en diferentes formatos y con diversos niveles de detalle. Más adelante estudiaremos otras formas de clasificarlos y expresarlos en formatos; pero por ahora nos concentraremos en una división fundamental: casos con formato **de alto nivel** y **expandido**.

6.10.1 Formato de alto nivel

Un **caso de uso de alto nivel** describe un proceso muy brevemente, casi siempre en dos o tres enunciados. Conviene servirse de este tipo de caso durante el examen inicial de los requerimientos y del proyecto, a fin de entender rápidamente el grado de complejidad y de funcionalidad del sistema. Estos casos son muy sucintos y vagos en las decisiones de diseño.

6.10.2 Formato expandido

Un **caso de uso expandido** describe un proceso más a fondo que el de alto nivel. La diferencia básica con el caso de uso de alto nivel consiste en que tiene una sección destinada al *curso normal de los eventos*, que los describe paso por paso. Durante la fase de especificación de requerimientos, conviene escribir en el formato expandido los casos más importantes y de mayor influencia; en cambio, los menos importantes pueden posponerse hasta el ciclo de desarrollo en el cual van a ser abordados.

6.11 Los sistemas y sus fronteras

Un caso de uso describe la interacción con un “sistema”. Las fronteras ordinarias del sistema son:

- la frontera hardware/software de un dispositivo o sistema de cómputo
- el departamento de una organización
- la organización entera

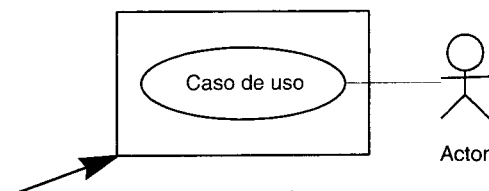


Figura 6.4 Frontera de un caso de uso.

Es importante definir la frontera del sistema para identificar lo que es interno o externo, así como las responsabilidades del sistema. El ambiente externo está representado únicamente por actores.

Estudiaremos un ejemplo de la influencia que tiene seleccionar la frontera del sistema: los pagos en la terminal del punto de venta y la tienda. Si elegimos como “el sistema” la tienda entera o el negocio (figura 6.6), el único actor es el cliente y no el cajero, porque este último es un recurso del sistema del negocio que realiza las funciones. Pero si escogemos como sistema el hardware y el software de la terminal del punto de venta (figura 6.5), se trata como actores al cliente y al cajero.

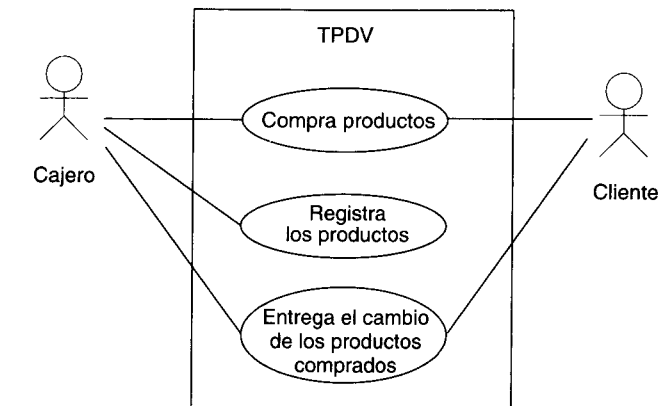


Figura 6.5 Casos de uso y actores cuando el sistema TPDV es la frontera.

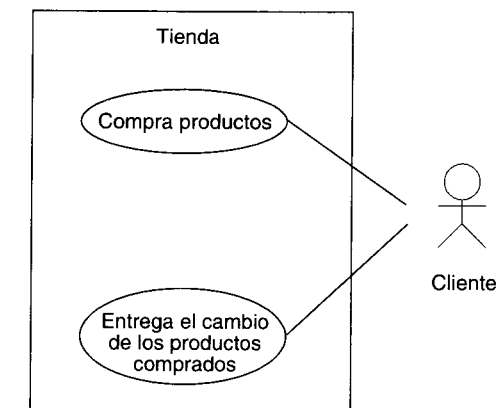


Figura 6.6 Casos de uso y actores cuando tienda es la frontera.

En la selección de una frontera del sistema influyen las necesidades de la investigación. Si estamos desarrollando un software de aplicación o un dispositivo, será razonable establecer la frontera del sistema en la del hardware y en la del software; por ejemplo, la terminal del punto de venta y sus programas constituye “el sistema”, y el cliente y el cajero son los actores (agentes) externos.

Si estamos efectuando la **reingeniería de procesos de la empresa** —reorganizando los procesos o la empresa para mejorar la competitividad o la calidad—, la selec-